

最佳 NIPT 时点选择与胎儿异常判定

摘要

无创产前检测 (NIPT) 是一种通过检测胎儿游离 DNA 片段来筛查胎儿染色体异常的重要技术。本文基于多时点随访数据构建了一个统一且可加权的建模与决策框架。

针对问题一, 为了量化 Y 染色体与孕周数、BMI 等指标的关系, 本文先利用**聚类自助法**稳健计算出加权 spearman 相关系数初步了解相关性, 随后结合 **GAMM 与 LMM** 建立 Y 染色体浓度与孕周和 BMI 的函数关系。结果显示, Y 染色体浓度与孕周为**显著正相关** (系数 0.10), 与 BMI 为**显著负相关** (系数 -0.17), GAMM 相比 LMM 对数似然显著提升, AIC 从 793.11 降至 755.96, 说明**非线性项**的必要性。

针对问题二, 为了降低孕妇潜在风险, 本文首先采用线性插值和问题 1 得到的 GAMM 模型估计出孕妇的**首次达标时间** T_i , 随后在 BMI 轴上, 以**最小化段内代价**为目标建立**动态规划**模型分段, 并用 BIC 自动选择最优组数 K^* , 在每个 BMI 组内, 对 T_i 近似出**首检达标曲线**, 定义**预期风险函数** $R_g(t)$, 再以 $R_g(t)$ 最小化为目标得到该组的**最优时点** t_g^* 。考虑检测误差时, 引入误差修正达标率, 并重复优化得到 $t_{g,err}^*$ 。最终, 将 BMI 区间分为 6 组, 分别约为 [20.7, 29.1], [29.1, 30.1], [30.1,31.2], [31.2, 32.6], [32.6, 33.4], [33.5, 46.9], 为各组修正后的最佳 NIPT 时点分别为 15.9 周, 18.3 周, 14.3 周, 16.0 周, 14.8 周, 19.1 周。

针对问题三, 为了在考虑**多种因素**的情况下使孕妇风险最小, 本文在原来的 GAMM 模型基础上加入年龄和相对 BMI **正交变化**后的身高与体重, 据此计算出达标曲线 $F_i(t)$, 接着在 BMI 升序上做 DP 动态规划分段, 并以在每段上“原始成本 + 结构惩罚”的风险最小为目标求出每组的最佳时点。最终, 本文得到 BMI 最优分组数为 3 组, 区间分别约为 [20.7, 30.6], [30.7,31.3],[31.3,46.9], 每组的最佳 NIPT 时点分别为 15.5 周, 16.25 周,17.75 周。

针对问题四, 为了构建高精度的女胎异常判定模型, 本文建立分组感知 (按孕妇) + 加权 (ω_q) 的**梯度提升决策树分类模型**, 联合**多路重采样**处理极端不平衡, 对预处理后的 552 条数进行了系统的特征工程、采样处理和参数优化, 采用 **5 折交叉验证**评估模型性能, 最终在 smoteenn 采样下, 在测试集上取得召回率 97.59%, F1 分数 93.61%, $AUC = 0.9742$, 精确率 89.95% 的优异性能, 在女胎异常判定上显著优于其他处理方法。

关键词: NIPT 加权混合效应模型 GAMM 动态规划 累计达标曲线

一、问题的重述

无创产前检测 (NIPT) 是一种通过采集母体血液检测胎儿游离 DNA 片段分析胎儿染色体是否异常从而达到产前诊断的技术方法, 其核心判断依据为胎儿性染色体浓度: 若孕期 10-25 周内, 男胎 Y 染色体浓度达 4% 及以上、女胎 X 染色体浓度无异常则 NIPT 结果基本准确, 否则认为结果不具有准确性, 需要进一步检测; 胎儿异常发现时间与风险相关, 12 周以内发现风险较低, 13-27 周风险较高, 28 周以后风险极高。男胎 Y 染色体浓度与孕妇孕周数、身体质量指数 (BMI) 密切相关^[7]。基于上述信息以及附件所包含的某地区孕妇的 NIPT 数据, 本文需要解决以下问题:

问题一 深入分析男胎孕妇中胎儿 Y 染色体浓度与孕周数、BMI 等指标之间的相关特性, 构建能够量化所述变量关联的关系模型, 并通过统计学方法检验该模型的显著性, 以验证模型对变量间关系刻画的可可靠性与有效性。

问题二 男胎孕妇的 BMI 是影响胎儿 Y 染色体浓度达标时间的主要因素。对男胎孕妇的 BMI 进行合理分组, 明确每组具体的 BMI 区间, 同时结合尽可能早地发现胎儿异常的目标, 确定每组对应的最佳 NIPT 时点, 并分析检测误差对分组结果的影响。

问题三 男胎 Y 染色体浓度达标时间不仅受 BMI 影响, 还与孕妇身高、体重、年龄等多种因素相关。现综合考虑这些因素, 结合检测误差以及胎儿 Y 染色体浓度达标比例, 对男胎孕妇的 BMI 进行合理分组, 确定每组的最佳 NIPT 时点。最后同样需分析检测误差对分组合理性的影响。

问题四 女胎孕妇的 21 号、18 号、13 号染色体非整倍体判定结果为核心依据, 综合考虑 X 染色体及上述三种染色体的 Z 值、GC 含量、读段数及相关比例、孕妇 BMI 等因素, 构建女胎异常判定方法。

二、问题分析

首先对附件的数据进行预处理, 优化数据格式、处理缺失值和异常值, 并且在考量测序质量后对相应数据优化和定权, 以抑制低质量数据对后续问题分析的影响。

2.1 问题一的分析

问题一要求分析男胎 Y 染色体浓度与孕妇孕周数、BMI 等指标的相关性, 建立相应的关系模型并检验显著性。首先, 考虑对某些孕妇有多次采血多次测量的情况, 直接计算的相关系数于 p 值偏乐观, 本文引入用**聚类自助法**计算斯皮尔曼相关系数的策略。建模阶段, 先使用**线性混合模型 LMM**, 再进一步采用**广义加性混合模型 GAMM**, 通过似然比检验比较模型优劣, 并输出显著性检验结果、效应方向和部分效应曲线, 最后结合残差诊断和灵敏度分析验证模型稳健性, 从而完成建模与求解。

2.2 问题二的分析

问题二聚焦于男胎孕妇的 BMI 对 Y 染色体浓度最早达标时间的影响，要求给出合理 BMI 分组和最佳检测时点，并分析检测误差对结果的影响。首先，利用线性插值和广义加性混合模型 GAMM，针对不同情况估计孕妇“**最早达标时间**” T_i 。再以加权负对数似然最小建立动态规划模型，同时结合 BIC 自动选择出组数 K^* 。再基于每组内 T_i 构造达标时间分布，定义**综合风险函数**，以风险最小为目标在合理搜索窗内求最优 NIPT 时点。最后把测量误差与个体差异纳入修正，得**误差修正**后的最优时点。最后做参数敏感性检查以分析检测误差对结果的影响。

2.3 问题三的分析

问题三进一步综合孕妇身高、体重、年龄等**多重因素**，同时考虑检测误差和浓度达标比例。首先进一步优化 GAMM 模型，综合多因素，估计个体在不同孕周下的**达标概率曲线**。然后以此为基础，利用**动态规划方法**在 BMI 轴上进行分段，结合综合风险函数求解合理的组数。最后在每个分组内确定能使风险函数最小化的最佳检测时点，从而得到**兼顾多因素差异与风险平衡**的分组和检测策略。

2.4 问题四的分析

问题四转向女胎的异常判定，关键是如何利用 13 号、18 号、21 号 X 染色体 Z 值、GC 含量、读段数及比例等多维信息进行综合判断，实际上是一个**二分类问题**。首先为避免同一孕妇的多次检测数据被同时划入训练集和测试集，本文先按孕妇分组把数据训练集和测试集，随后做**特征工程与数据清洗**，筛选并标准化数值特征，得到核心候选特征组。对异常样本的**不平衡问题**，本文尝试构造多种**重采样版本**：随机过/欠采样、SMOTE、Borderline-SMOTE、SMOTEENN。然后基于特征数据，尝试两种**数集成模型** Gradient Boosting、AdaBoost 与六种数据集分别搭配。最后均衡各项指标最佳选择总体最佳模型，利用 **k 折交叉验证**模型表现。最终可得判定异常女胎的模型方法^[8]。

三、模型的假设

- 同一孕妇的多次检测中，测序平台的检测精度保持一致，无因仪器校准偏差导致的批次效应；
- Y 染色体浓度检测值与真实值的偏差（残差 ε_{ij} ）、孕妇个体差异导致的 Y 浓度波动（随机截距 u_{0i} ）均服从正态分布，即 $\varepsilon_{ij} \sim N(0, \sigma_\varepsilon^2)$ 、 $u_{0i} \sim N(0, \sigma_u^2)$ ，且两者相互独立；
- 在足够窄的 BMI 区间内， T_i 近似服从正态分布且可按权重 w 估计段内参数；
- 在误差修正过程中，所有孕妇的测量误差方差结构一致。

四、 符号说明 ^[9]

符号	定义说明
$i = 1, \dots, N$	孕妇个体索引, N 为样本个体数
$j = 1, \dots, m_i$	第 i 个个体的检测索引, m_i 为检测次数
t_{ij}	第 i 个体第 j 次检测的孕周
b_{ij}	对应检测时的 BMI; 以该个体首次检测的 BMI 作为该人的代表 BMI, 记为 b_i
$y_{ij} \in (0, 1)$	对应检测的 Y 染色体浓度 (比例)
τ	达标阈值, 设定为 $\tau = 0.04$
$w_{Q,ij}$	某次检测的质量权重
$w_i = \sum_{j=1}^{m_i} w_{Q,ij}$	个体权重
T_i	个体最早达标时间
$\mathcal{G} = \{G_1, \dots, G_K\}$	按 BMI 把样本分成 K 个相邻分段, 组内样本索引集合为 G_g
$\hat{F}_g(t)$	组 g 内 T_i 的加权核化经验 CDF
$R_g(t)$	在时点 t 进行 NIPT 的组别风险函数
t_g^*	组 g 的最优 NIPT 时点
$t_{g,err}^*$	在误差修正后的最优时点 ^[9]

五、 数据预处理与分析

附件所给数据出现异常、缺失、数据格式不统一等问题，如 A139 个体 F 栏末次月经日期缺失、B007 个体怀孕日期早于末次月经日期、H 栏检测日期数据有“年月日”与“年-月-日”两种格式，等等。为确保本次建模求解的准确性和有效性，本文首先对数据进行以下预处理：

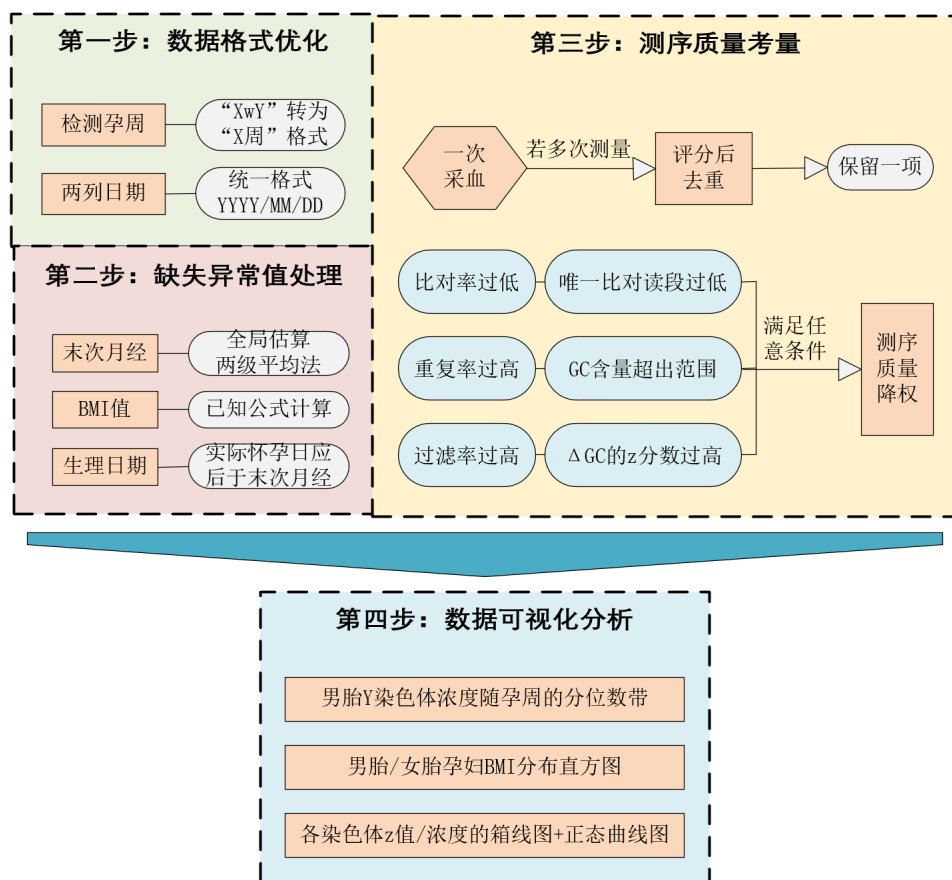


图 1 数据预处理

5.1 数据清洗与补充

(1) 日期格式统一与变换：将 F 栏末次月经与 H 栏检测日期对应数据格式统一转换为年-月-日；孕周数据格式从 “Week w + Day” 转换为 “x 周”，转换公式为 $x = \text{Week} + \frac{\text{Day}}{7}$ ；

(2) 去掉 BMI 缺失数据组，去掉染色体 Z 值缺失数据组；

(3) 日期异常值删除：我们将实际怀孕日期定义为检测日期减去检测孕周，发现部分数据的实际怀孕日期早于末次月经记录日期，这可能和临床实际情景有关，鉴于建模与分析的严谨性，本文删除了这些日期异常值组对应数据；

(4) 末次月经缺失值补全：我们根据全表数据，对非缺失组按 检测日期-检测孕周对应天数-

末次月经 计算均值用于推测缺失值^[10]。

5.2 测序质量评估

软门控判定是筛选优质样本的核心步骤，本文预处理依次检测以下规则：若比对比例低于比对比例的 5% 分位数 (M5)、重复读段比例高于重复读段比例的 95% 分位数 (N95)、过滤读段比例高于被过滤读段数比例 95% 分位数 (AA95)、唯一比对读段数低于唯一比对读段数的 5% 分位数 (O5)，或整体 GC 含量超出 [P5, 0.60] 范围，亦或任一染色体 ΔGC (为该染色体 GC 含量与整体 GC 含量差值) 的 Z 值绝对值大于 3，则判定样本测序质量不合格，并记录具体的风险类型。最后统计每条样本的风险项数量以区分样本质量等级。

对于被软门控的样本，程序会进一步计算质量评分 Q：

$$Q = \max \left(0, (f_{\text{reads}} \times f_{\text{align}} \times f_{\text{dup}} \times f_{\text{filt}} \times f_{\text{gc_overall}} \times f_{\text{delta}})^{1/6} \right) \quad (1)$$

其中各参数定义如下¹

$$\begin{aligned} f_{\text{reads}} &= \min \left(\frac{\text{唯一比对读段数}}{O_5}, 1.0 \right) \\ f_{\text{align}} &= \min \left(\frac{\text{比对率}}{M_5}, 1.0 \right) \\ f_{\text{dup}} &= \max \left(1.0 - \frac{\text{重复率}}{N_{95}}, 0.0 \right) \\ f_{\text{filt}} &= \max \left(1.0 - \frac{\text{过滤率}}{AA_{95}}, 0.0 \right) \\ f_{\text{gc_overall}} &= \begin{cases} \frac{GC_{\text{overall}}}{GC_{\text{LOW}}} & , \text{若 } GC_{\text{overall}} < GC_{\text{LOW}} \\ 1.0 - \frac{GC_{\text{overall}} - GC_{\text{HIGH}}}{1.0 - GC_{\text{HIGH}}} & , \text{若 } GC_{\text{overall}} > GC_{\text{HIGH}} \\ 1.0 & , \text{其他情况} \end{cases} \\ f_{\text{delta}} &= \max \left(1.0 - \frac{Z_{\text{max}}}{3.0}, 0.0 \right) \end{aligned}$$

该评分综合考虑多维度指标：通过读段数量、比对比例、重复读段比例、过滤读段比例在对应分位数区间内的位置，转换为 0-1 的标准化得分；对整体 GC 含量超出合理范围的样本，采用温和衰减策略将其得分映射至 0.6-1.0 区间；对 ΔGC 偏差的样本，取 13/18/21 号染色体偏差得分的最小值，最终通过几何平均计算 Q 值，再根据 GAMMA (1.0) 计算加权得分；其余样本的样本则默认权值为 1.0，无需计算 Q 值。

针对孕妇一次抽血多次检测的情况，本文按未被软门控优先 $\rightarrow$ 权重越高越优 $\rightarrow$ 比对

¹ GC_{LOW} 为整体 GC 含量的 5% 分位数; GC_{HIGH} 为固定阈值 0.60; Z_{max} 为 13/18/21 号染色体 ΔGC 的 Z 分数绝对值的最大值

比例越高越优 → 唯一读段数越高越优 → 重复率 / 过滤率越低越优 → ΔGC 最大绝对偏差越小越优 → 检测日期越早越优的优先级排序，仅保留每组中质量最优的 1 条样本，最终将筛选后的样本与关键指标整合。

5.3 数据可视化

为了从不同维度挖掘胎儿 Y 染色体浓度与孕妇孕周、BMI 等等关键性指标的关联规律，基于预处理后的男胎孕妇数据，本文首先运用 Origin 程序进行如下的可视化分析。

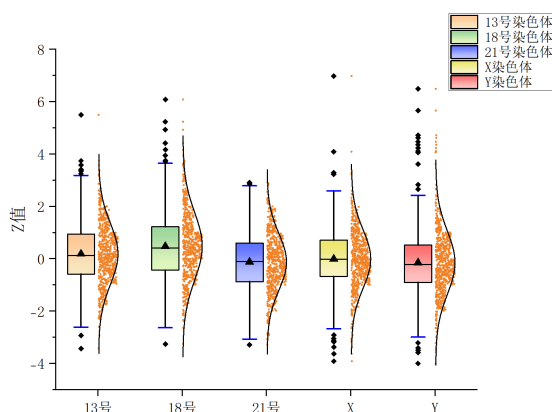


图 2 染色体 Z 值箱线图

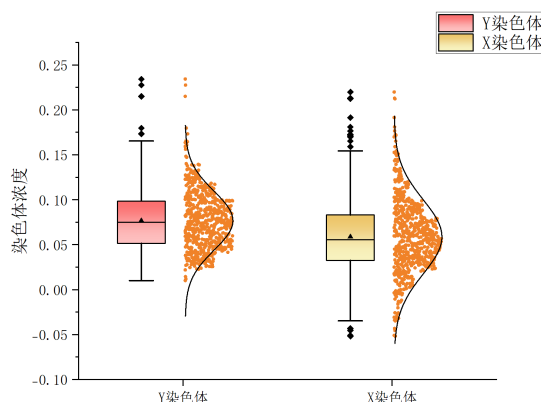
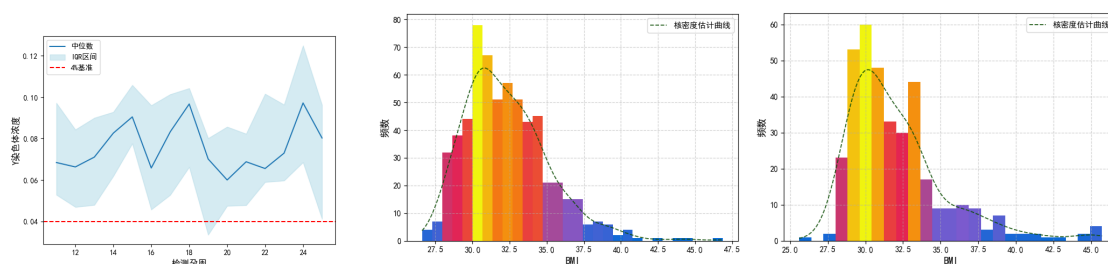


图 3 性染色体浓度箱线图

我们研究男胎的染色体数据并绘制了箱线图2、3，初步分析数据特征。13 号、18 号、21 号染色体的 Z 值分布整体近似对称，中位数与零轴接近，四分位间距较小。其中 21 号最为收敛、18 号略显离散，离群点数量有限，说明三种染色体样本的总体均处于统计正常区。对比之下，X 与 Y 染色体的 Z 值波动更大，点云与核密度曲线均显示尾部较厚，说明在个体差异与测序波动的影响下，性染色体读数差异化更明显。Y 浓度总体高于 4% 门槛，但仍有约八分之一样本未达标，这说明按孕妇 BMI 和检测孕周分层优化检测时点具有必要性。



(a) 胎儿 Y 染色体浓度-孕周的分位数带 (b) BMI 频数分布直方图 (男胎) (c) BMI 频数分布直方图 (女胎)

图4a分周计算 Y 浓度分位数，其中位数水平可观察到整体有波动，提示孕周以外因素

也在对 Y 染色体浓度的变化起作用。阴影带宽度在不同孕周数区间不等，直接用“周”作分组做决策需谨慎。四分位距（IQR）区间大部分位于 4% 之上，对于确定男胎最佳 NIPT 检测时点提供初步方向。从中位数层面看，折线水平远高于 4%，但 IQR 带逼近甚至略低于 4%，说明仍有相当比例样本可能不达标，这种风险随周数起伏。图4b与图4c用 Silverman 法则自动选择带宽，平滑直方图并叠加一条趋势曲线。趋势图线单峰、右偏，统一时点检测会让一部分高 BMI 人群更容易早测失败或者低于阈值，应按 BMI 分层选择检测时点。

六、模型的建立与求解

6.1 问题一模型建立与求解

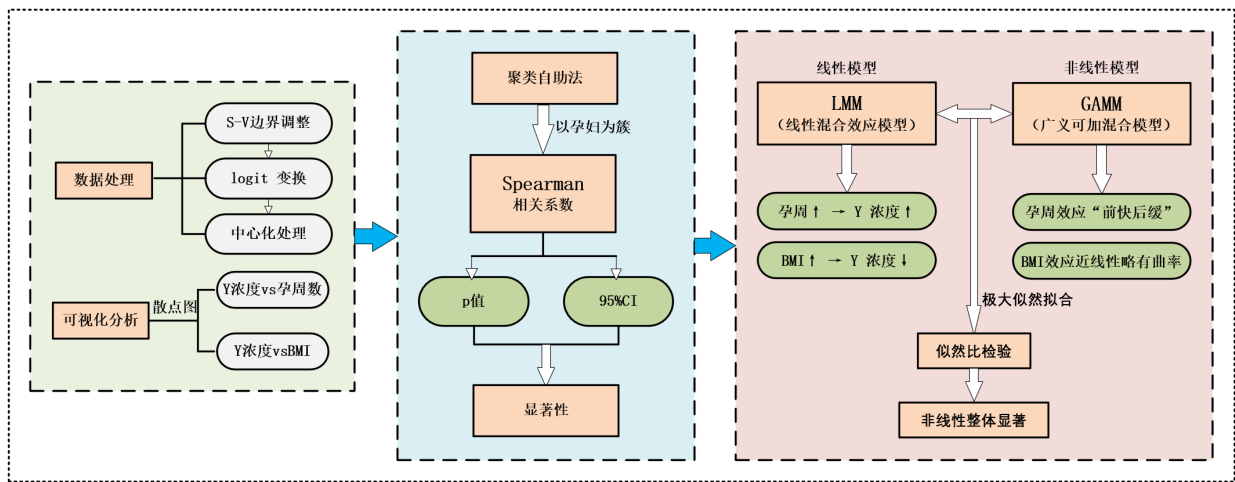


图 4 问题一处理流程

问题一的核心目标是在考虑存在同一孕妇重复测量的情况，刻画 Y 浓度与孕周、BMI 的函数关系，给出具有显著性水平的结论，在此基础上本文给出部分可视化结果。为分析胎儿 Y 染色体浓度与孕妇孕周数及 BMI 的关系，我们首先采用 Spearman 相关并以孕妇为簇进行自助法估计 95% CI 与显著性；随后构建广义加性混合模型，因 $Y \in [0, 1]$ 采用 Smithson–Verkuilen 调整与 logit 链接。固定效应强制包含孕周与 BMI（并在扩展模型中加入年龄），以似然比检验方法（LRT）检验显著性。考虑测序质控指标，我们构造主成分质量得分以控制技术噪声，并在敏感性分析中比较删除 GC 范围在 40% - 60 % 之外的样本影响。进一步，我们将 Y 4% 视为达标事件，拟合混合效应逻辑回归并报告 AUC 与校准情况。

6.1.1 探索性分析

我们首先对题目指明的 BMI 与孕周数两个指标进行分析，对相应数据数据进行筛选提取，分别作出其与 Y 染色体浓度的散点图5,6，该图显示 Y 浓度与孕周关系并非简单线性，后续将孕周作为主要平滑变量纳入模型以更准确刻画趋势。

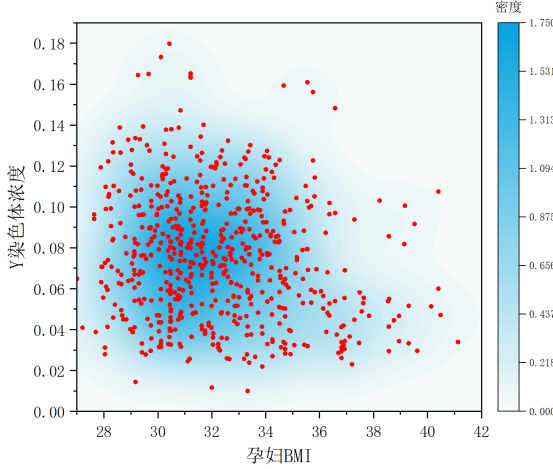


图 5 BMI - Y 染色体浓度散点图

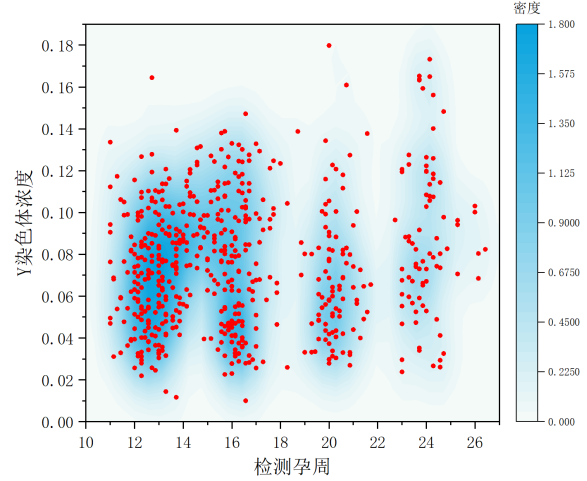


图 6 孕周数 - Y 染色体浓度散点图

6.1.2 聚类自助法计算带权 Spearman 相关系数

聚类自助法 (Cluster Bootstrap) 是一种在统计分析中用于处理数据聚类结构的重采样技术，核心是对数据中的聚类单元进行有放回地抽样，生成生成规模相当的重采样数据集，基于数据集重复执行分析后整合结果评估原数据集的稳定性，解决了传统聚类结果不稳定、可信度难评估的问题，最终输出更稳健、可解释的聚类结论，本文以孕妇为单位进行有放回抽样，完整保留抽中聚类单元内的全部观测值构建自助样本，并重复抽样 - 构建样本过程 1000 次 (本文设定)，通过多次计算相关系数得到得到相关系数 $\{r_{s,b}^{(w)}\}_{b=1}^{1000}$ 的总体分布，计算出置信区间、标准误等统计量，p 值按分布相对 0 的双尾概率给出。

Spearman 相关系数 (Spearman's Rank Correlation Coefficient) 是一种非参数统计方法，用于衡量两个变量之间的单调相关关系，且对极端值和测序误差噪声的敏感度较低。但由于同一孕妇有多次记录，普通 Spearman 相关系数检验未考虑组内观测值相关、组间独立的特性，计算出的 p 值会偏小。考虑到数据记录有质量差异，用 w_i 加权能够抑制某些低质观测值的影响，据此，带权 Spearman 相关系数计算方式如下，对两个变量 X, Y 做平均秩: $R_X = \text{rank}(X)$, $R_Y = \text{rank}(Y)$. 带权 Spearman 相关系数为:

$$r_s^{(w)} = \frac{\sum_i w_i (R_{X,i} - \bar{R}_X^{(w)}) (R_{Y,i} - \bar{R}_Y^{(w)})}{\sqrt{\sum_i w_i (R_{X,i} - \bar{R}_X^{(w)})^2} \sqrt{\sum_i w_i (R_{Y,i} - \bar{R}_Y^{(w)})^2}}. \quad (2)$$

其中 $\bar{R}^{(w)} = \frac{\sum w_i R_i}{\sum w_i}$, w_i 为质量权重; $R_{X,i}, R_{Y,i}$ 为第 i 个观测的秩; $\bar{R}_X^{(w)}$ 为 R_X 的加权均值。

计算结果如表??和图7、8所展示，Y 染色体浓度和表??中指标有关，结合 p 值，应优先选择 BMI、检测孕周、X 染色体浓度、18 号染色体 Z 值、在参考基因组上比对的比例等指标，这些变量与 Y 染色体浓度的关联经统计检验可靠，具有中等以上的相关线性强度，其中与 18 号染色体 Z 值、BMI、参考基因组比对比例来呈现负相关关系，与 X 染色体浓度、检测孕周呈现正相关关系，这些是模型的核心解释变量；可排除的无关变量：如 21 号染色体 GC 含量、GC 含量等，相关系数绝对值接近 0 且不显著，纳入模型会增加复杂度

且无实际解释力。

表 1 指标与相关系数表

指标	相关系数	指标	相关系数	指标	相关系数
X 染色体浓度	0.464	Y 染色体 Z 值	0.114	13 号染色体 Z 值	-0.075
18 染色体 Z 值	-0.176	重复读段比例	0.0690	18 号染色体 GC 含量	-0.0098
孕妇 BMI	-0.166	原始读段比例	-0.0022	X 染色体 Z 值	-0.0518
参考基因组比对比例	-0.155	过滤读段数比例	0.0092	13 号染色体 GC 含量	-0.0354
年龄	-0.123	检测孕周	0.0819	21 号染色体 Z 值	0.0370

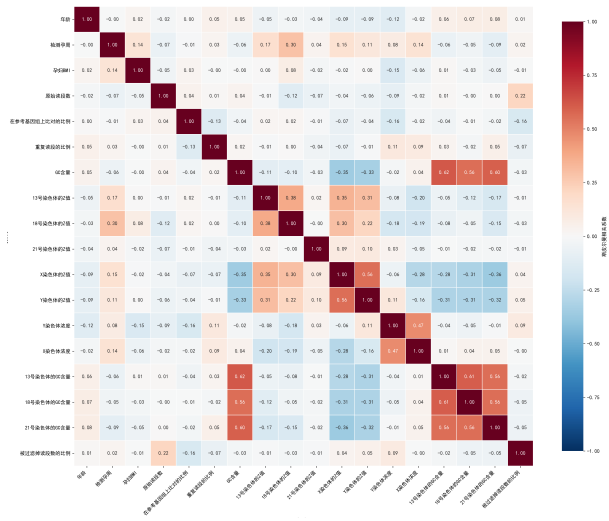


图 7 带权 Spearman 相关系数热力图

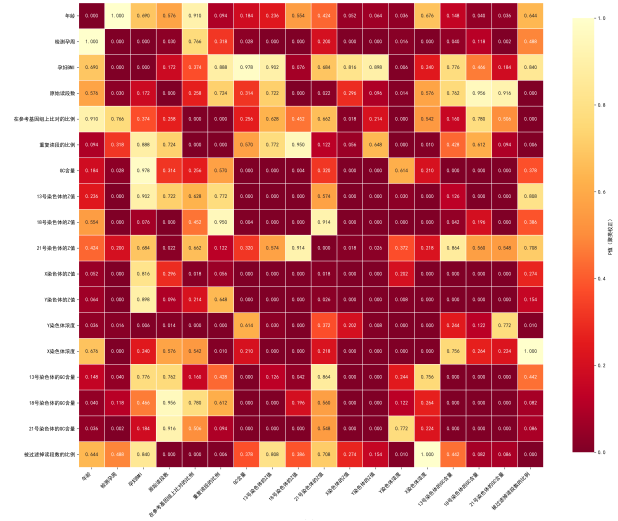


图 8 p 值热力图

6.1.3 广义可加混合模型 (GAMM) 构建

首先由于 Y 染色体浓度 $Y \in [0, 1]$, 属于比例型数据, 存在边界问题。采用 Smithson–Verkuilen 进行边界调整后 logit 变换:

$$Y^* = \frac{Y \cdot (n - 1) + 0.5}{n}, \tilde{Y} = \text{logit}(Y^*). \quad (3)$$

其中 n 为非缺失样本数. 门槛 $y^\dagger = 4\%$ 相应变换为在 $\eta^\dagger = \text{logit}\left(\frac{y^\dagger(n-1)+0.5}{n}\right)$.

其次, 强制包含孕周与 BMI, 并在扩展模型中加入年龄。因为前期相关性分析已初步表明这些指标与 Y 染色体浓度相关, 将其作为固定效应纳入模型, 可量化它们对 Y 染色体浓度的影响。为提高模型收敛性, 作中心化如下减去样本均值的中心化处理:

$$GA_c = GA - \overline{GA}, BMI_c = BMI - \overline{BMI}, Age_c = Age - \overline{Age}. \quad (4)$$

最后还需引入孕妇层随机截距 $u_{0i} \sim N(0, \sigma_u^2)$, 是因为同一孕妇的多次检测记录存在相关

性，随机截距体现了不同孕妇之间的个体差异。综合以上考虑，本文建立如下广义可加混合模型：

$$\tilde{Y}_{ij} = \alpha + f_1(GA_{c,ij}) + f_2(BMI_{c,ij}) + \beta_{Age}Age_{c,ij} + u_{0i} + \varepsilon_{ij}. \quad (5)$$

其中 f_1, f_2 为自然三次样条 $f_k(x) = \sum_{\ell=1}^{d_k} \theta_{k\ell} b_{k\ell}(x)$, $k \in \{1, 2\}$. 基函数纬度为 d_1, d_2 .

为了检验非线性模型的必要性，本文同时拟合了线性 LMM，模型形式为：

$$\tilde{Y}_{ij} = \alpha + \beta_{GA}GA_{c,ij} + \beta_{BMI}BMI_{c,ij} + \beta_{Age}Age_{c,ij} + u_{0i} + \varepsilon_{ij}. \quad (6)$$

其中 f_1, f_2 约束为线性。

通过设定不同的自由度 ($d_1, d_2 \in \{4, 5, 6\} \times \{4, 5, 6\}$)，并采用网格搜索结合 AIC 准则²选择最优自由度，可使平滑函数既能充分拟合数据的非线性特征，又能避免过度拟合。

6.1.4 最大似然估计 (ML) 求解模型

似然函数构建

假设随机效应 u_i 和残差 ε_{ij} 独立，观测数据的联合对数似然函数为：

$$\ell(\theta) = \sum_{i=1}^m \ln \left[\int \prod_{j=1}^{n_i} f(\text{logit}Y_{ij} | u_i; \theta) \cdot f(u_i; \sigma_u^2) du_i \right] \quad (7)$$

其中： $\theta = (\beta_0, \beta_1, \beta_2, \sigma_u^2, \sigma_\varepsilon^2)$ 为 GAMM 待估参数向量； $f(\text{logit}Y_{ij} | u_i; \theta)$ 为条件正态密度； $f(u_i; \sigma_u^2)$ 为随机效应的正态密度。

LBFGS 算法最大化对数似然

ML 的目标是找到使 $\ell(\theta)$ 最大的参数 $\hat{\theta}$ ：

$$\hat{\theta} = \arg \max_{\theta} \ell(\theta).$$

LBFGS 算法核心是通过近似 Hessian 矩阵指导参数更新。

Step1) 梯度计算与 Hessian 矩阵近在当前参数 θ_k 处，计算对数似然函数的梯度：

$$g_k = \nabla \ell(\theta_k) = \left[\frac{\partial \ell}{\partial \beta_0}, \frac{\partial \ell}{\partial \beta_1}, \frac{\partial \ell}{\partial \beta_2}, \frac{\partial \ell}{\partial \sigma_u^2}, \frac{\partial \ell}{\partial \sigma_\varepsilon^2} \right]^T \quad (8)$$

Step2) 通过历史迭代信息参数更新量 $\Delta\theta_k = \theta_{k+1} - \theta_k$ 和梯度变化 $\Delta g_k = g_{k+1} - g_k$ 近似 Hessian 矩阵的逆 H_k^{-1} ：

$$H_{k+1}^{-1} = V_k^T H_k^{-1} V_k + \rho_k \Delta\theta_k \Delta\theta_k^T$$

²AIC (Akaike Information Criterion, 赤池信息准则)，是一种用于评估统计模型拟合优良性的标准。其定义公式为

$$AIC = 2k - 2\log(L)$$

，其中 k 是模型中参数的数量， L 是似然函数值。AIC 越小，模型拟合越优。

其中 $\rho_k = 1/(\Delta g_k^T \Delta \theta_k)$, V_k 为校正项。

Step3) 参数更新与收敛搜索方向

- 由近似的 H_k^{-1} 和梯度确定下降方向: $d_k = H_k^{-1} g_k$ 。
- 步长选择: 通过线性搜索确定步长 α_k , 使 $\ell(\theta_k + \alpha_k d_k)$ 增大。
- 收敛判断: 当梯度的 L_2 范数小于阈值 (如 10^{-5}) 时停止迭代: $\|g_k\| < \epsilon$ 。

6.1.5 求解结果与显著性检验

我们对比了 LMM 和 GMM 两种模型的结果:

LMM 模型求解结果显示: $Z \sim 1 + \text{GA}_c + \text{BMI}_c + \text{Age}_c$, 随机截距: 孕妇, 其边际 $R^2 = 0.1609$, 条件 $R^2 = 0.9230$; 随机截距方差 0.185, 残差方差 0.0577, 这说明固定效应 (GA/BMI/ 年龄) 解释了约 16% 的总体变异, 而加入个体差异后总体解释度很高; 孕妇间的基线差异显著存在。效应方向: $\beta_{\text{GA}} > 0$ (每 +1 周孕周提高 Y 的 logit, OR ≈ 1.049); $\beta_{\text{BMI}} < 0$ (每 +1 BMI 降低 Y 的 logit, OR ≈ 0.973)。等效孕周: $\theta = -\beta_{\text{BMI}}/\beta_{\text{GA}} \approx 0.575$ 周, 意为 BMI 每增加 1 个单位, 需顺延 ≈ 0.58 周孕周才能抵消其负向影响 (95% CI 参见运行日志)。

最优 GAMM: $1 + \text{cr}(\text{GA}_c, \text{df} = 5) + \text{cr}(\text{BMI}_c, \text{df} = 6) + \text{Age}_c$ 。拟合优度: $\ell = -363.980$, AIC = 755.96; 相较线性 LMM ($\ell = -391.555$, AIC = 793.11), 改进显著 (LRT = 55.15, $\text{df} = 9$, $p = 1.14 \times 10^{-8}$)。这说明非线性项是必要的。

通过似然比检验, 可判断允许孕周与 BMI 呈平滑函数是否能显著提升模型拟合效果。表2展示了两组模型的对照结果, 线性 LMM 的对数似然值 $\log L = -412.004$, AIC = 834.01; GAMM 的 $\log L = -391.451$, AIC = 806.90, 计算得到似然比³LR = 41.106, 自由度 $\text{df} = 7$, $p = 7.73 \times 10^{-7}$, 远小于常用的显著性水平 ($p = 0.05$), 说明孕周与 BMI 呈平滑函数关系。因此, GAMM 能显著提升模型拟合效果。

表 2 模型比较结果

模型	logL	AIC	LRT
LMM	-412.004	834.01	—
GAMM	-391.451	806.90	—
差异统计量	—	—	LR = 41.106 ($\text{df} = 7$, $p = 7.73 \times 10^{-7}$)

6.1.6 结果可视化与解释

在分析某一变量对 Y 染色体浓度的影响时, 将另一变量固定在中位数水平, 可排除该变量的干扰, 更清晰地展示单一变量对 Y 染色体浓度的独立影响, 如图10与图9所示, 为了展示不确定性, 图中添加了 95% 置信带, 置信带越窄, 说明模型在该区间的预测越准确。

从整体上看, Y 染色体浓度随孕周上升而上升, 曲线在 10–16 周上升更快, 其后增幅趋缓, 孕周的增加对 Y 染色体浓度的提升作用更明显, 这为早检 + 必要复检的流程设计提供了定量依据, 对于需要尽早检测但 Y 染色体浓度尚未达标的孕妇, 可在中后期适当缩短复检间隔, 以尽早得到 Y 染色体浓度达标的时点。在 BMI 指标方面: 整体为负向, 曲线

³LR = $2(\log L_{\text{GAMM}} - \log L_{\text{LMM}})$

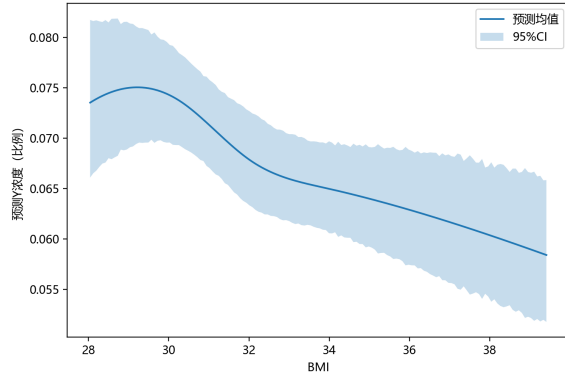


图 9 自然样条 GAMM 的 BMI 部分效应

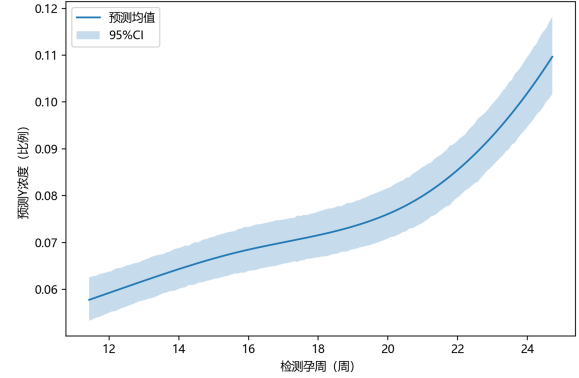


图 10 自然样条 GAMM 的孕周部分效应

接近线性, BMI 越高, 胎儿 Y 染色体浓度越低, 验证了高 BMI 可能抑制 Y 染色体浓度升高这一临床经验, 因此, 高 BMI 群体选择更晚的检测时点, 避免过早检测导致的结果不准确。

6.2 问题二模型建立与求解

问题二的核心目标是通过 BMI 分组确定每组最佳检测时点, 实现潜在风险最小化的目标优化。基于问题一所建立的最优 GAMM 模型, 本文所采取的方法是先计算每位孕妇的 Y 染色体浓度首次达标时间 T_i , 再以 BMI 为依据对 T_i 进行最优分组, 最后通过风险函数找到每组最佳检测时点, z 最后分析检测误差的影响。

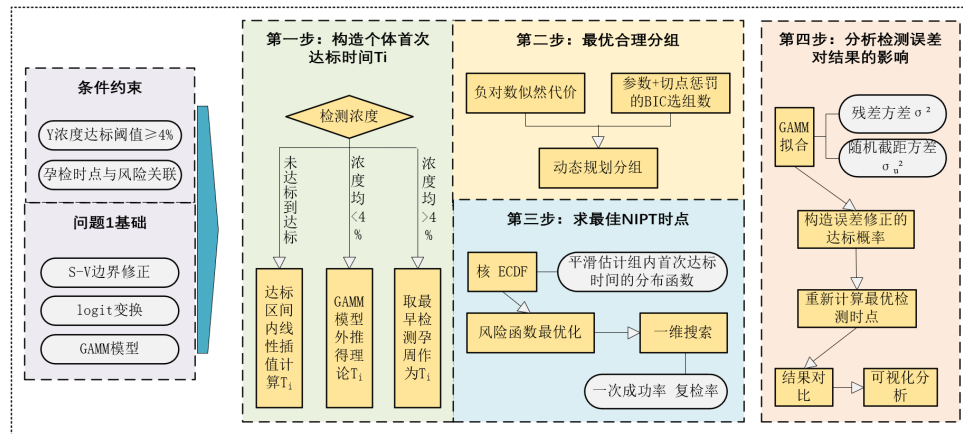


图 11 问题二处理流程

• **个体首次达标时间 T_i 的确定** 首先筛选附件数据, 保留孕妇代码、检测孕周、BMI、Y 染色体浓度 4 类核心列 (若有年龄数据则额外保留)。由附件资料, 可知孕妇检测情况按自身检测结果相互对比可分为两种情况, 相应的 T_i 确定方法列举如表3所示⁴:

⁴若外推 $T_i < 10$ 周则取 10 周作为 T_i 。

表 3 不同情况下 T_i 的计算方法

检测记录情况	计算方法
浓度：未达标 $\rightarrow$ 达标	在达标孕周区间内线性插值计算 T_i
检测浓度均 $< 4\%$ 或均 $\geq 4\%$	用 GAMM 模型外推得到理论 T_i

6.2.1 BMI 轴加权动态规划

在 NIPT 时点选择中，传统经验分组（如将 BMI 按固定等距区间划分）是常用方法，但该方法一方面忽略了数据的内在分布规律，某区间内 T_i 可能差异较大，不应归为一组，另一方面也可能导致各等距区间样本量失衡，基于以上考虑，本文以 T_i 的实际分布规律为依据，通过统计方法找到“组内差异小、组间差异大”的 BMI 分组，确保每组孕妇的达标时间特征一致。把所有孕妇按 BMI 升序排成 $i = 1, \dots, N$ ，把 BMI 轴切成若干区间，那么动态规划的目标是：在固定组数 k 时，找到把序列划成 k 段的切法，使组内代价之和最小。

• **分组合理性量化** 对于按 BMI 升序后的索引区间 $[i, j]$ ，（含样本 $\{z_u\}_{u=i}^j$ 服从正态分布 $z_u \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mu_g, \sigma_g^2)$ ）。以个体权重 w_i 计算加权均值与方差： $\mu = \frac{\sum_{r=i}^j w_r x_r}{\sum_{r=i}^j w_r}$ ， $\sigma^2 = \frac{\sum_{r=i}^j w_r (x_r - \mu)^2}{\sum_{r=i}^j w_r}$ ， $x_r = \log T_r$ 。段的最小负对数似然：那么，该段最小负对数似然（NLL）为

$$\mathcal{C}(i, j) \doteq -\log L(\mu, \sigma^2) = \frac{W}{2} [1 + \log(2\pi\sigma^2)], \quad W = \sum_{r=i}^j w_r. \quad (9)$$

计算得到某一 BMI 区间作为一组的代价，NLL 越小，说明该区间内孕妇的 T_i 分布越集中，分组越合理。

• **动态规划全局最优分段** 动态规划的核心逻辑是递推寻找最小总代价。令 $\text{DP}_k[j]$ 为“将前 j 个样本切成 k 段”的最小总代价， $\text{DP}_1[j] = \mathcal{C}(1 : j)$ 。建立动态规划模型：

$$\text{DP}_k[j] = \min_{i \in \{k, \dots, j\}} \{\text{DP}_{k-1}[i-1] + \mathcal{C}(i : j)\}. \quad (10)$$

遍历所有可能的分组切点⁵，保存回溯指针即可得到在给定 k 下的全局最优切点集合 $\{c_1, \dots, c_{k-1}\}$ 。BIC 用有效样本量 $W_{\text{tot}} = \sum_i w_i$ ： $\text{BIC}(K) = 2 \sum_{g=1}^K \mathcal{C}_g + 2K \log W_{\text{tot}}$ ，取 BIC 最小的 K^* 。求解结果如图12所示。

6.2.2 最佳 NIPT 检测时点建模

• **核 ECDF 估计组内达标分布** 为了用非参数方法估计每组孕妇的达标概率随孕周变化的规律，从而为风险函数计算提供概率基础，本文利用核 ECDF 通过高斯核函数对经验累计分布函数 ECDF 进行平滑，得到连续、单调的 t 时点达标概率 $\hat{F}_g(t)$ 。令 $\mathcal{I}_g = [c_{g-1}, c_g)$ ， $g = 1, \dots, G$ ，为 G 个连续区间， $S_g = \{i : \text{BMI}_i \in \mathcal{I}_g\}$ 为第 g 组样本集。在得到每组的 $\{T_i\}_{i \in S_g}$

⁵该动态规划算法的时间复杂度是 $O(kN^2)$ ，这样的复杂度是可以承受的。

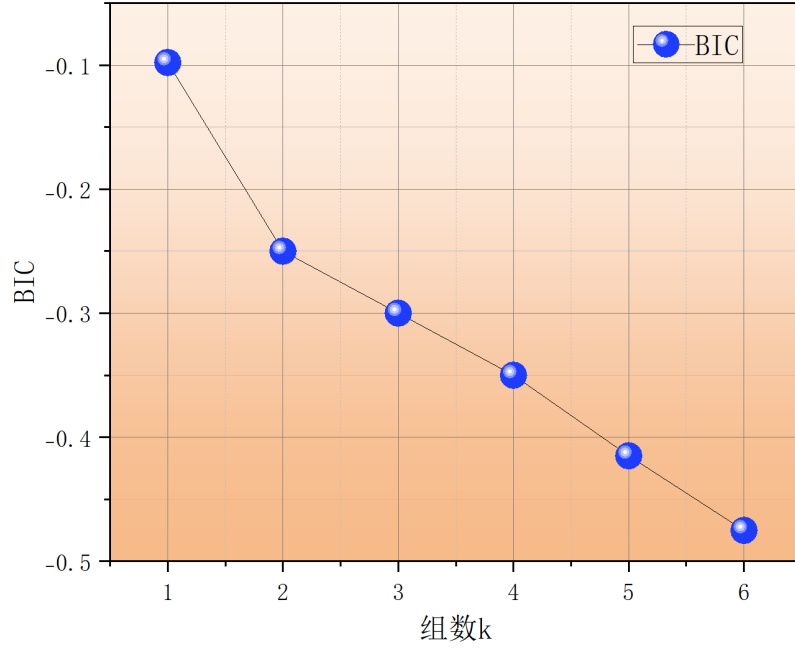


图 12 BIC 随组数变化折线图

后:

$$\hat{F}_g(t) = \frac{1}{n_g} \sum_{i \in S_g} \Phi \left(\frac{t - T_i}{h_g} \right), \quad h_g = 1.06 s_g n_g^{-1/5}. \quad (11)$$

其中 Φ 是标准正态分布函数, s_g 为 $\{T_i\}$ 的样本标准差。该 $\hat{F}_g$ 严格单调非降、在全域 $[0, 1]$ 有界, 适合做风险积分。

• **风险函数优化** 设首检时点 $t \in [10, 25]$, 未达标则每隔 $\Delta > 0$ 周复检: $t_k = t + k\Delta$, $k = 0, 1, 2, \dots$, $\lambda \geq 0$ 为首检失败惩罚, 那么: 对于第 g 组, 期望风险函数为

$$\mathcal{R}_g(t) = \sum_{k=0}^{\infty} w(t_k) \left[\hat{F}_g(t_k) - \hat{F}_g(t_{k-1}) \right] + \lambda \left[1 - \hat{F}_g(t) \right]. \quad (12)$$

约定 $t_{-1} = 0$ 。最优检测时点:

$$t_g^* = \arg \min_{t \in [10, 25]} \mathcal{R}_g(t). \quad (13)$$

利用 python 程序在 $t \in [10, 25]$ 上一维搜索最小的点。对每组给出 t_g^* 、一次成功率 $\hat{F}_g(t_g^*)$ 、复检率 $1 - \hat{F}_g(t_g^*)$ 以及 $\mathcal{R}_g(t_g^*)$ 。

6.2.3 求解过程与结果展示

- Step 1 读数与质控汇总 $\rightarrow$ 汇出个体权重 w_i ;
- Step 2 拟合带权 GAMM, 得到 $\hat{\sigma}_s^2, \hat{\sigma}_t^2$;

- Step 3 计算每位孕妇的 T_i 并保存；
- Step 4 BMI 按升序排列，做 DP+BIC 分段选出最优组数 $\hat{G}^*$ 与切点；
- Step 5 对每组计算 $\hat{F}_g(t)$ 并最小化 $R_g(t)$ 得到 t_g^* ，同时给出一次达标率” $\hat{F}_g(t_g^*)$ 与复检率 $1 - \hat{F}_g(t_g^*)$ 。

求解结果图13和14所示。曲线特征反映分组达标规律，BMI 较小组曲线上升越陡峭，该组孕妇在对应孕周区间内达标概率增长快，整体达标时间更早。通过对比不同组曲线的差异，可验证 BMI 分组的有效性。低 BMI 组的核 ECDF 曲线上升早、达标概率高，最优时点更早；高 BMI 组曲线上升晚、达标概率增长慢，最优时点更晚，符合 BMI 越高，Y 染色体浓度达标越慢的临床规律。结合曲线可直接读取每组的一次达标率（最优时点对应的纵轴值）、复检率，为临床检测资源分配提供数据支持。求解的各组划分情况以及相应最优

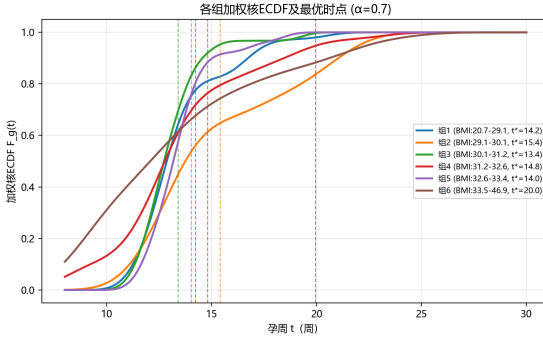


图 13 各组核 ECDF 以及最优时点

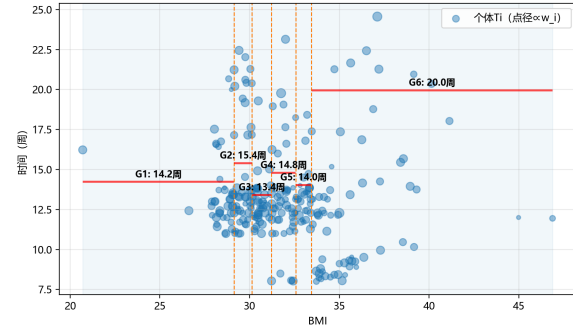


图 14 BMI 划分散点图 (最优检测时间水平)

NIPT 罗列如下第一 BMI 组划分区间为 20.7 ~ 29.1, 最优 NIPT 时点为第 14.2 孕周，第二 BMI 组划分区间为 29.1 ~ 30.1, 最优 NIPT 时点为第 15.4 孕周；第三 BMI 组划分区间为 30.1 ~ 31.2, 最优 NIPT 时点为第 13.4 孕周；第四 BMI 组划分区间 31.2 ~ 32.6, 最优 NIPT 时点为 14.8 孕周；第五 BMI 组划分区间为 32.6 ~ 33.4, 最优 NIPT 时点为第 14.0 孕周；第六 BMI 组划分区间为 33.5 ~ 46.9，最优 NIPT 时点为第 20.0 周。

6.2.4 检测误差影响分析

实际检测中存在测序失败、不确定因素影响，量化误差的影响。本文通过 GAMM 模型的随机截距方差和残差方差，计算考虑误差后的个体达标概率，再通过组内平均得到误差修正后的组内达标概率。

问题一 GAMM 在 logit 尺度上的线性预测模型为 $\hat{\eta}_i(t) = \mu_i(t) + u_i + \varepsilon_{it}$ ，其中个体随机截距 $u_i \sim \mathcal{N}(0, \sigma_u^2)$ ，残差 $\varepsilon_{it} \sim \mathcal{N}(0, \sigma_e^2)$ 。对给定个体 i 与时点 t ，其一次达标的边际概率近似为

$$p_i(t) = \int \text{logit}^{-1}(\mu_i(t) - \eta^\dagger + \sigma Z) \phi(Z) dZ \approx \sum_{q=1}^Q w_q \text{logit}^{-1}(\mu_i(t) - \eta^\dagger + \sigma \sqrt{2} x_q), \quad (14)$$

其中 (x_q, w_q) 为 Q 点 Hermite 节点与权重， $\sigma^2 = \sigma_u^2 + \sigma_e^2$ 。取 $Q = 20 \sim 50$ 足够稳

定。组内的平均边际概率曲线：

$$\bar{p}_g(t) = \frac{1}{n_g} \sum_{i \in S_g} p_i(t). \quad (15)$$

将 $\bar{p}_g(t)$ 单调化得到 $\bar{F}_g^{(\text{err})}(t)$ 作为误差-修正的累计达标函数，重算期望风险函数以及最优检测时点：

$$\mathcal{R}_g^{(\text{err})}(t) = \sum_{k \geq 0} w(t_k) [\bar{F}_g^{(\text{err})}(t_k) - \bar{F}_g^{(\text{err})}(t_{k-1})] + \lambda [1 - \bar{F}_g^{(\text{err})}(t)], \quad (16)$$

$$t_{g,\text{err}}^* = \arg \min_{t \in [10, 25]} \mathcal{R}_g^{(\text{err})}(t). \quad (17)$$

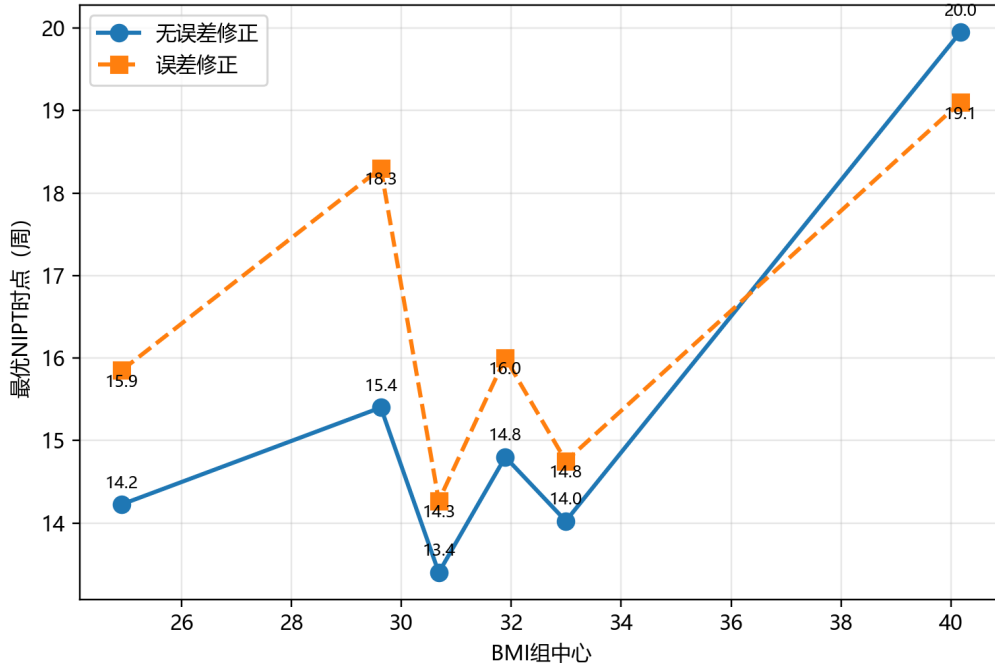


图 15 BMI 与最优 NIPT 时点折线图（加权）

图15比较了各 BMI 分组中心的未修正达标概率曲折线与误差修正后的折线，实线代表展示了各组 BMI 中心群体及其最优 NIPT 时点，虚线加入了检测误差和个体差异。从图形上看，在考虑误差后，虚线整体上移，除了最高 BMI 一组 NIPT 时点有所前移外，最优 NIPT 时点均有不同程度推迟。据此，分组策略应当结合误差修正。

6.3 问题三模型建立与求解

问题三的核心目标是综合考虑 BMI、孕周、年龄、身高、体重等多因素影响，建立多目标优化模型，为男胎孕妇的 NIPT 检测提供最优分组策略和检测时点建议。整体解决思路是基于问题二构建的广义可加混合模型，充分估计各因素对 Y 染色体浓度的影响，并综合估计结果的影响，给出以 BMI 为分组依据的检测策略。

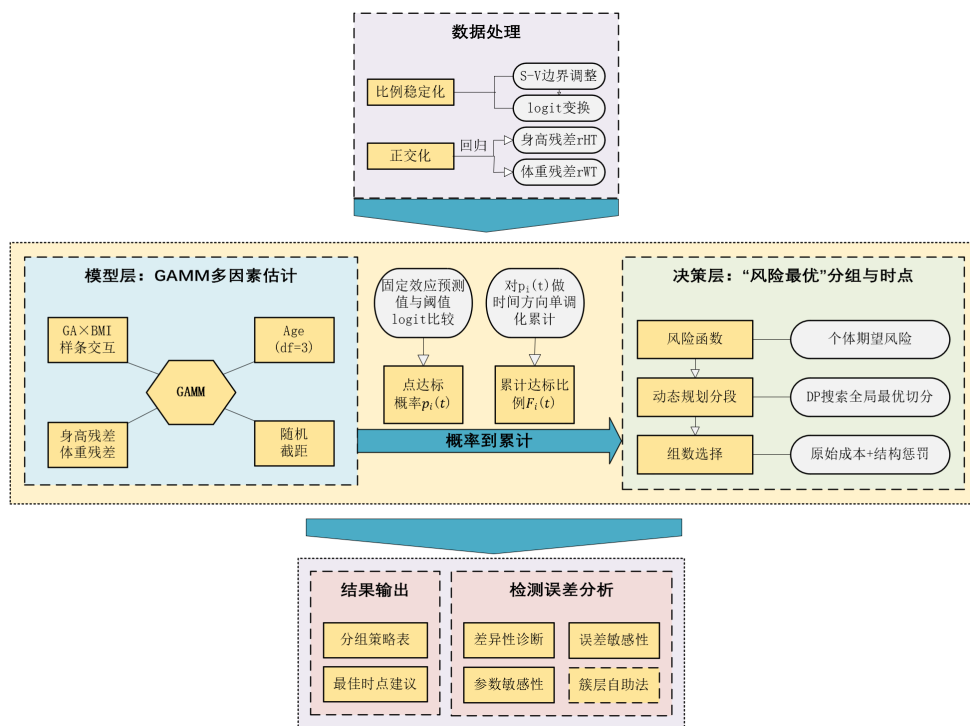


图 16 问题三求解流程

6.4 数据可视化与作图分析

图17的每个小图展示两个变量的散点关系，直观显示 BMI、孕周、体重、身高、年龄、GC 含量等变量之间的相关性与分布形态。部分变量关系近似线性（如 BMI 与体重），部分存在非线性与离群点，揭示了多因素间的潜在的相互关系。整体上，在 BMI 为 25-40 之间，样本分布趋于集中，特别是在 BMI 为 30-35 区间，体重主要分布在 0.5 至 1.0 的范围内，反映了研究样本中较高 BMI 群体的特征。进一步分析显示，Y 染色体浓度与 BMI 呈负相关关系，较高的 BMI 通常伴随较低的 Y 染色体浓度，这为 NIPT 的时点选择提供了重要数据支持。

通过小提琴图19,20的分析,可以看出数据中的体重、孕妇 BMI、13 号、18 号和 21 号染色体的 Z 值呈现出不同程度的分布特点。BMI 的数据分布相对较为集中,而染色体的 Z 值则显示出不同样本间的基因波动,尤其在 13 号染色体 Z 值上最为明显。结合年龄、身高等变量的分布情况,BMI 和染色体 Z 值之间的潜在关系值得进一步分析,以确定影响 NIPT 准确性的关键因素。

6.4.1 身高体重的正交化

由于 BMI 与身高 (HT)、体重 (WT) 存在强相关性 ($BMI \approx WT/HT^2$)，若直接纳入模型会导致共线性，干扰 BMI 的核心效应。因此需要分别计算两指标对 BMI 的残差，然后剥离 BMI 对两指标的解释部分（正交化处理）：

$$\begin{aligned} rHT_i &= \log(HT_i) - \hat{\mathbb{E}}[\log(HT) | \log(BMI)]_i. \\ rWT_i &= \log(WT_i) - \hat{\mathbb{E}}[\log(WT) | \log(BMI), rHT]_i. \end{aligned} \quad (18)$$

使 rHT 和 rWT 保留了独立于 BMI 的身材信息，既补充了变量维度，又避免干扰 BMI 的核心作用。

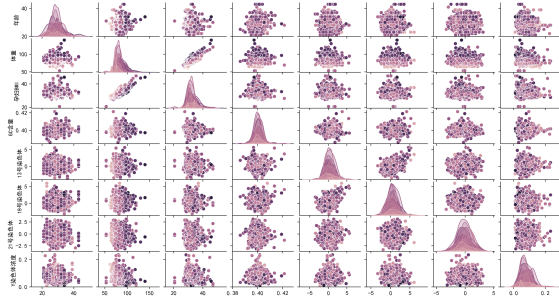


图 17 各指标间散点图

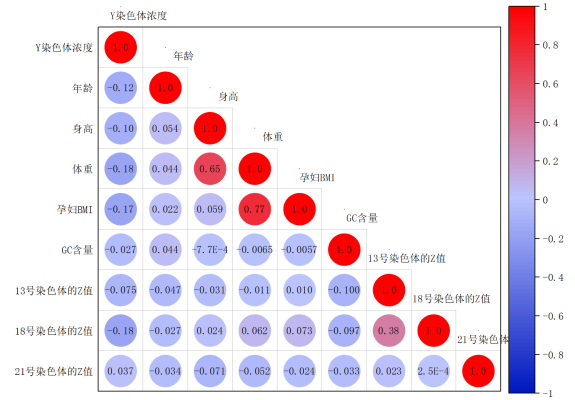


图 18 各指标间热力图

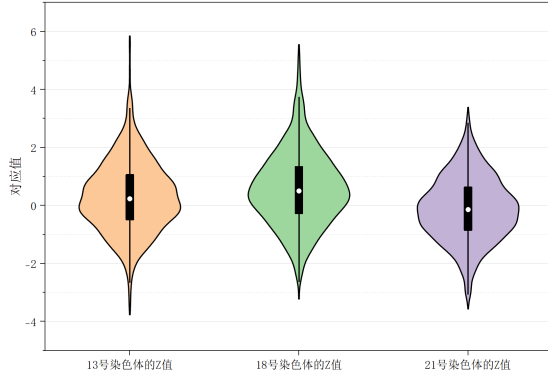


图 19 13、18、21 号染色体小提琴图

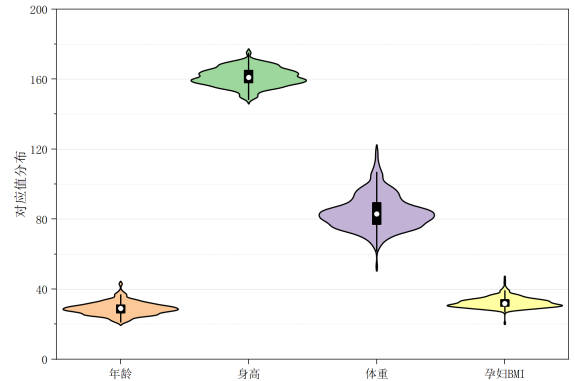


图 20 孕妇各指标小提琴图

6.4.2 GAMM 模型构建

基于问题一所构建的原来的 GAMM 模型，本部分上新增年龄和相对 BMI 正交变化后的身高与体重变量构建 GAMM 模型，其固定效应包括模型的群体平均趋势 $\mu_i(t)$ ，由多个平滑函数 $f_k(\cdot)$ 组合而成，形式为 $\mu_i(t) = \underbrace{f_1(GA_c) \times f_2(BMI_c)}_{\text{主交互}} + f_3(rHT_c) + f_4(rWT_c) + f_5(Age_c)$ ，其中各 $f_k(\cdot)$ 为三次回归样条 (Age 取低自由度以抑制过拟合)，“ $\cdot_c$ ”代表中心

化。随机截距：

$$\text{logit}(Y_i^*(t)) = \mu_i(t) + b_i + \varepsilon, \quad b_i \sim \mathcal{N}(0, \sigma_b^2), \varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2).$$

• **达标概率与累计曲线** 设定达标阈值 $\theta = 0.04$ ，个体在时刻 t 的一次达标概率 $p_i(t) = \mathbb{P}(Y_i(t) \geq \theta) \approx \sigma(\mu_i(t) - \ell_\theta)$ ；累计达标曲线为 $F_i(t) = \max_{s \leq t} p_i(s)$ 表示截至时刻 t ，个体能达到的最大达标概率。为了让结果更稳健，引入总方差 $\sigma^2 = \sigma_b^2 + \sigma_\varepsilon^2$ ，重新计算一次达标概率 $p_i^{(\text{err})}(t) = \mathbb{P}(\mu_i(t) + Z \geq \ell_\theta) = 1 - \Phi\left(\frac{\ell_\theta - \mu_i(t)}{\sigma}\right)$ ，最后再基于 $p_i^{(\text{err})}(t)$ 计算累计达标曲线 $F_i^{(\text{err})}(t) = \max_{s \leq t} p_i^{(\text{err})}(s)$ 。

6.4.3 动态规划最优分组算法

基于 GAMM 模型输出，计算个体条件达标概率 $p_i(t) = \text{logit}^{-1}(\mu_i(t) - \eta^\dagger)$ 及其累计形式 $F_i(t) = \max_{s \leq t} p_i(s)$ 。定义风险函数 $\mathcal{R}_i(t) = \sum_{k=0}^{\infty} w(t_k) [F_i(t_k) - F_i(t_{k-1})] + \lambda [1 - F_i(t)]$ ，其中权重参数取 $(W_1, W_2, W_3) = (0, 2, 10)$ ，首检失败成本 $\lambda = 0.5$ 。采用动态规划算法进行最优分割，以 $\text{Crit}(k) = \text{DP}_k[N] + \gamma(k-1) \log N$ 为选择准则 ($\gamma = 0.15$)，确保分组结果既最小化总体风险又控制模型复杂度。

6.4.4 求解结果

GAMM 残差方差 0.0625，随机截距方差 0.1554。结构惩罚下得到最优组数 $G^* = 3$ ，组间 BMI 边界与指标如下（表4）。组 1 与组 3 的最佳时点为 11.75 周，组 2 为 23.75 周；对应一次达标率分别为 0.613、0.710、0.589（重测率为 0.387、0.290、0.411）。

表 4 分组检测相关指标

组别	人数	BMI 范围	最优检测时点 (周)	一次达标率	重测率
1	136	20.70 ~ 31.34	11.75	0.6124	0.3876
2	51	31.40 ~ 33.29	23.75	0.7103	0.2897
3	80	33.05 ~ 46.87	11.75	0.5886	0.4114

6.5 问题四模型建立与求解

问题四的核心目标是完成一个二分类任务，标签 0 代表染色体正常，1 代表 21/18/13 号染色体非整倍体异常，本文综合考虑 X 染色体及目标染色体的 Z 值、GC 含量、读段数比例、孕妇 BMI 等多维度特征，基于机器学习模型构建女胎异常判断方法，解决数据不平衡、同一孕妇多检测记录等关键问题。

6.5.1 数据预处理与探索性分析

- **特征筛选** 排除标签列、ID 列等非数值列，仅保留 Z 值、GC 含量、BMI、读段数等可量化的数值列同时过滤非空值比例 $\geq 30\%$ 的特征，从而去除冗余信息和低质量特征降低模型复杂度，提升泛化能力。
- **缺失处理与标准化** 对所有数值特征做中位数填充并标准化至零均值单位方差。避免均值填充对极端值敏感的缺点，同时保留特征的分布趋势。

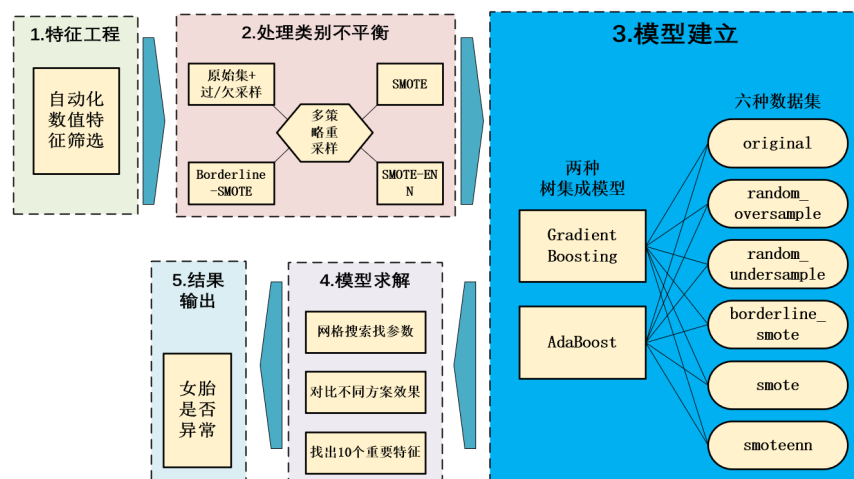


图 21 问题四解决流程

• **样本质量权重** 由于本题异常样本数据偏少,可通过赋予其更高权重,让模型在训练时更重视异常样本,避免模型因正常样本多而造成偏差。若预处理阶段未给出权重时,置 $w_{Q,i} = 1$ 。训练时本文采用加权经验风险,评估时同时报告未加权与按 w_Q 加权两套指标,以体现样本质量差异对结论的影响。

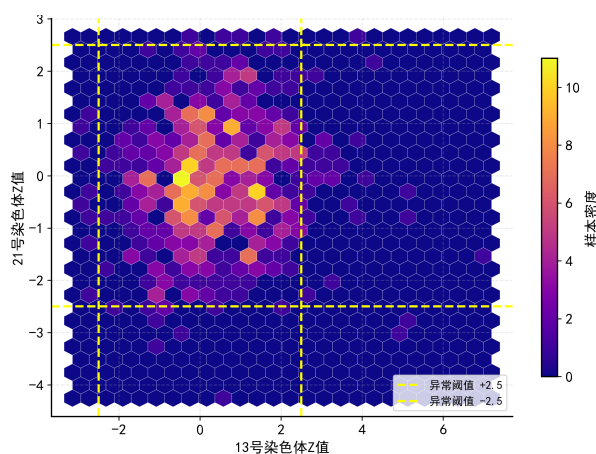


图 22 13、18 号染色体 Z 值热力图

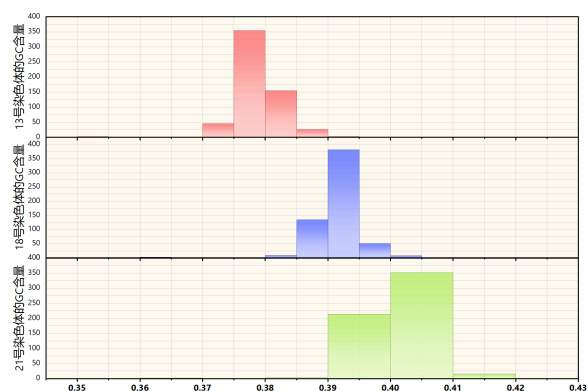


图 23 GC 浓度多面板直方图

热力图22展示了女胎孕妇 13 号和 21 号染色体 Z 值的二维分布特征。由图可知大部分样本集中在 Z 值 ± 2 范围内的正常区域,符合标准正态分布特征。红色虚线标记 ± 2.5 异常阈值,显示异常样本主要分布在四个象限的边缘区域。正常染色体的 Z 值分布在原点附近,为后续异常判定模型提供了重要的数据基础。GC 含量分布分析图23展示了三个关键染色体的 GC 含量分布特征。13 号染色体 GC 含量主要分布在 0.37-0.39 区间,18 号染色体分布在 0.38-0.40 区间,21 号染色体分布在 0.39-0.42 区间。灰色阴影区域标记了 0.40.6 这一正常 GC 含量范围,各染色体 GC 含量的差异分布为异常判定提供了重要的质量控制指标。

6.5.2 分组切分数据与重采样策略

同一孕妇的多次检测若被拆分到训练/测试两侧，会因模型见过该孕妇的分布特征而抬高测试指标。为此我们按孕妇代码做分组随机切分，避免信息泄露。对同一孕妇的多个样本，取多数类标签（如某孕妇 3 次检测中 2 次为正常，则该孕妇的标签为“正常”）作为分层依据。接着，按孕妇的聚合标签分层抽样，确保训练集 / 测试集的异常率与原数据保持一致。

本体异常样本数据稀少，若直接训练模型，模型会偏向预测正常，准确率高但召回率低，导致漏检异常胎儿，为此，本文通过重采样策略平衡类别分布。我们在训练集内部对比以下重采样：随机过采样、随机欠采样、SMOTE、Borderline-SMOTE、SMOTEENN。为避免同一孕妇跨集合的信息泄漏，采用 GroupKFold/按孕妇代码分组的切分方式。重采样仅在训练折内部执行，验证折与测试集保持原始分布不做重采样。分组与加权口径见综合分析报告“按孕妇分组”“使用样本权重 (w_Q)”说明。

6.5.3 分类器

本文使用两种集成学习模型（GBDT 梯度提升决策树、AdaBoost 自适应提升），基于不同重采样数据集训练模型，输出对异常的后验概率估计 $\hat{p}_i$ ，使用网格搜索遍历树数量、树深度等参数网格，以加权 AUC 为评分指标，筛选最优模型参数。

- **梯度提升树 (GBDT)** 通过迭代训练多棵弱决策树，每棵树拟合前序模型的残差，最终通过加权投票输出预测结果。其损失函数通常为对数损失，数学上通过梯度下降最小化损失。
- **自适应增强 (AdaBoost)** 通过迭代调整样本权重，错分样本权重增加，正确样本权重减少，每棵树基于加权样本训练，最终通过树的性能权重输出预测结果。
- **预测性能评价指标** 传统交叉验证（如 KFold）可能将同一孕妇的样本分到不同折，导致模型在验证集上的性能偏高，为了合理评估模型性能，本文采取 GroupKFold 方法，通过分组约束确保验证集的样本与训练集无关联，评估结果更合理。预测性能指标是衡量一个模型好坏的关键，本文采用的预测性能指标包括 AUC、AP、准确率、精确率、召回率、F1、特异度、Brier 分数。为了平衡异常样本，各指标计算均作加权处理。
- **模型选择** 超参数选择采用自定义加权交叉验证：在 GroupKFold 的每个验证折上获得 out-of-fold 概率 $\hat{p}$ ，用加权指标 $ROC-AUC_w = roc_auc_score(y, \hat{p}, sample_weight = w)$ ， $AP_w = average_precision(y, \hat{p}, sample_weight = w)$ 以及 $Brier_w = \frac{\sum w_i(\hat{p}_i - y_i)^2}{\sum w_i}$ 、 $F1_w$ 等进行评分，用同一套加权指标确定最优模型与阈值 τ^* 。最终测试集只评估一次，沿用验证集确定的 τ^* 。

6.5.4 结果与可视化

如图24比较了多种重采样策略，五张对比图里，几乎所有指标在 SMOTEENN 上都明显领先（AUC 1、F1 0.92–0.94、Recall 0.96–0.98、MCC 0.82–0.85），故将其作为首选方案。

最优组合如表5所示，若以筛查为目标，以少漏检为优先，则选 SMOTEENN + AdaBoost。它在测试集上 Recall 0.976、F1 0.936、Precision 0.899、MCC 0.854、AUC 0.974、AP 0.976，召回和 F1 都是最强，若以风险排序/打分为主，则选 SMOTEENN + GradientBoosting。它的 AUC 0.986、AP 0.990 为全局最佳，排序与区分能力最强；阈值后移也能得到与 AdaBoost 接近的召回。对于 NIPT 这类宁可复检也不能漏检的场景，应当选择

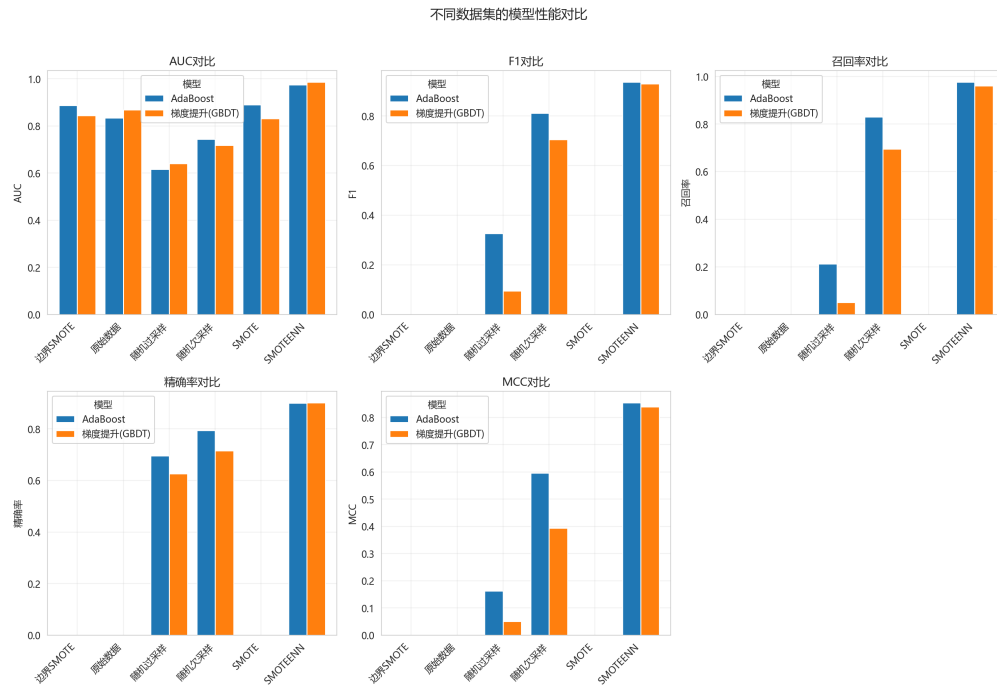


图 24 重采样策略下模型表现

SMOTEENN+AdaBoost。

表 5 最优模型组合

重采样	模型	Set	ACC	Prec	召回率	F1	AUC
Smoteenn	GradientBoosting	测试集	0.910	0.883	0.961	0.922	0.986
Smoteenn	GradientBoosting	训练集	1.000	1.000	1.000	1.000	1.000
Smoteenn	AdaBoost	训练集	0.993	0.991	0.996	0.993	0.998
Smoteenn	AdaBoost	测试集	0.926	0.899	0.976	0.936	0.974

图25对比了六种重采样方式与两类树模型的组合效果。其中 SMOTEENN 明显最优: AUC 分别约 0.986 (GradientBoosting) 与 0.974 (AdaBoost), 说明在考虑样本质量权重后仍能把阳性与阴性高度区分。SMOTE 与 Borderline-SMOTE 已比原始数据效果好, AUC 约 0.86–0.91, 表明仅靠过采样的几何插值就能缓解不平衡带来的学习偏置, 但它们的曲线在中高 FPR 区域仍有下垂, 提示边界仍混有噪声。Original 的 AUC 只有 0.83–0.86, 不平衡与噪声都未处理; Random undersample 因丢失多数类信息, AUC 仅 0.71–0.73; Random oversample 最弱, AUC 约 0.60–0.62, 曲线接近对角线, 易过拟合复制样本而对真实分布泛化差。

最佳模型计算出来的特征重要性如图26展示, 共筛选了 10 个在染色体异常判定排名靠前的因素。女性无 Y 性染色体, 其 X 染色体浓度的特征重要性较高, 远超位列第二的重读段的比例, 除此之外, 剩余的八个因素包括年龄、孕妇 BMI, 体重, 参考基因比例, 测序质量考量计数, X 染色体 Z 值, GC 含量与身高, 他们的重要性相互接近, 且水平较低。这种特征重要性模式为临床检测提供了关键指标的优先级排序参考意见。

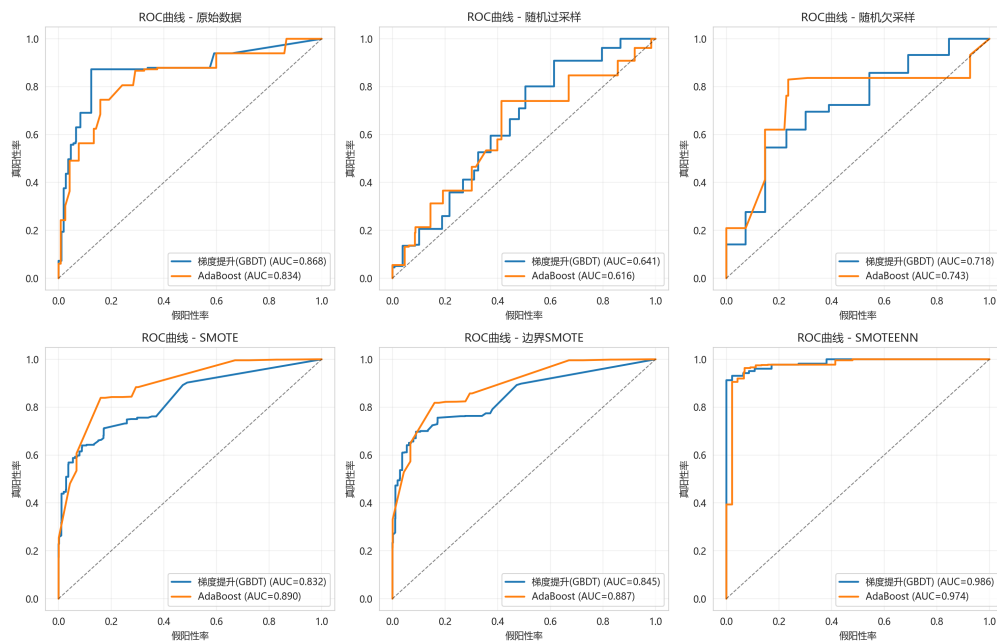


图 25 加权 ROC 曲线

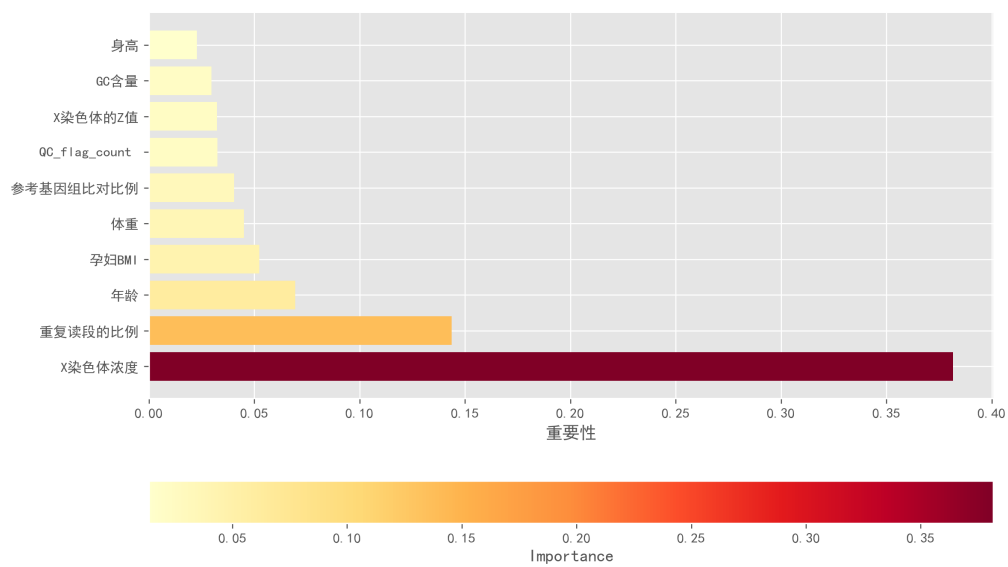


图 26 特征重要性

七、 模型的评价与推广

7.1 模型的优点

- 数据层的稳健性处理：提出“软门控 + 质量分数”机制，对读段数、比对率、重复率、过滤比例、整体 GC 与 ΔGC 多维指标进行柔性判定，并以几何平均（读段/比对加权权重）生成连续型质量权重；同孕妇同日仅保留“质量最好”的记录，降低重复与测序噪声带来的偏倚。
- 构建“带权 Spearman $\rightarrow$ 簇自助法”流程，既能兼顾样本权重，又能按“孕妇”为簇进行有放回重抽估计标准误与置信区间，稳健检验孕周/BMI 与 Y 浓度的单调相关性。
- 动态规划分段：以动态规划最小化“分段内风险 + 结构惩罚”的总准则，既自动确定组数与边界，又联动各组 $t \setminus t^*$ ；输出每组的规模、BMI 界、一次达标率与重测率等直观总结。
- 女胎异常判定的完整流水线：自动特征筛选与中位数插补、标准化；按“孕妇 ID”进行分组切分与 GroupKFold 交叉验证防止信息泄漏；结合随机过/欠采与 SMOTE/SMOTEENN 等缓解类别失衡；评估指标覆盖 AUC、F1、召回、MCC、特异性等。

7.2 模型的缺点

- GAMM 对初始值敏感，易陷入局部最优。
- 类别失衡与合成样本偏差：重采样（如 SMOTE/SMOTEENN）虽提高召回，但可能改变真实分布并引入合成噪声。
- 可解释性有限：女胎判定主要采用树集成（GBDT/AdaBoost），虽然性能稳健，但机制解释不如参数模型直观。

7.3 模型的推广

- 检测质量权重的数据驱动学习：在现有“阶段权重 + 体重指数调节”的基础上，引入真实复检成本、随访结局等信息，学习权重与惩罚强度；同时对“阶段积分索引”进行整体校准，使风险度量更贴近实际。
- 与生存/时间到事件方法衔接：把“首次达标”视作事件，采用离散时间危险率或加速失效时间模型，替代单一阈值穿越法，更好处理不规则观测与删失；据此将风险函数改写为基于事件概率的临床效用。
- 女胎判定的工程化与可解释化：在“分组交叉验证 + 重采样 + 网格搜索”框架上，补充阈值优化与概率校准（如对数几率回归校准、等温回归校准）；结合特征重要性与基于博弈论的贡献分解等工具，提升可解释性与阈值落地性。

参考文献

- [1] Zhang R, Zhang H, Li Y, et al. External quality assessment for detection of fetal trisomy 21, 18, and 13 by massively parallel sequencing in clinical laboratories[J]. The Journal of Molecular Diagnostics, 2016, 18(2): 244-252.
- [2] 蓝志衡.DNA 序列分布比率法在无创产前胎儿染色体非整倍体检测中的应用 [D]. 华南理工大学,2020.DOI:10.27151/d.cnki.ghnlu.2020.005132.
- [3] 袁玉英. 一种基于机器学习的无创产前筛查孕妇肿瘤模型构建及研究 [D]. 华南理工大学,2020.DOI:10.27151/d.cnki.ghnlu.2020.001545.
- [4] 郑芸芸, 万陕宁, 宋婷婷, 等. 8594 例孕妇无创 DNA 产前检测在胎儿染色体非整倍体疾病筛查中的临床应用 [J]. 实用妇产科杂志,2019,35(01):68-71.
- [5] 龙洋, 罗艳梅, 徐聚春, 等. 无创 DNA 检测在诊断高龄孕妇胎儿非整倍体中的应用 [J]. 实用妇产科杂志,2017,33(05):373-375.
- [6] 边旭明. 胎儿染色体非整倍体的无创 DNA 产前检测 [J]. 实用妇产科杂志,2013,29(05):330-333.
- [7] DeepSeek, DeepSeek-V3, 深度求索 (DeepSeek), 2025-09-04
- [8] ChatGPT, ChatGPT5-thinking, OpenAI , 2025-09-05
- [9] ChatGPT, ChatGPT5-thinking, OpenAI , 2025-09-06
- [10] 腾讯元宝, Hunyuan, 腾讯 (Tencent), 2025-09-07

附件 A: 支撑材料文件列表

文件名	简介
数据预处理.py	数据预处理相关代码实现
问题 1_ 相关分析.py	问题 1 的相关分析代码
问题 1_LMM.py	问题 1 的 LMM 模型实现代码
问题 1_GAMM.py	问题 1 的 GAMM 模型实现代码
问题 2.py	问题 2 的求解代码
问题 3.py	问题 3 的求解代码
问题 4.py	问题 4 的求解代码
AI 工具使用详情说明.pdf	本次数模比赛 AI 工具使用报告
GAMM_coef_table_weighted.xlsx	问题 1 GAMM 模型各指标信息
GAMM_model_summary_weighted.txt	问题 1 GAMM 建模过程信息报告
聚类自助法 _ 斯皮尔曼相关分析结果 _ 带权.xlsx	问题 1 斯皮尔曼相关分析表格
聚类自助法 _ 所有配对结果 _ 带权.xlsx	问题 1 所有配对结果表
聚类自助法 _ 显著配对结果 _ 带权.xlsx	问题 1 显著配对结果表
bic_table.txt	问题 2 BMI 最优分组决策数据
bmi_groups_results.csv	问题 2 最优分组的各组信息
run_summary.txt	问题 2 建模过程信息报告
Ti_table.csv	问题 2 个体首次达标时间信息表
groups_policy_improved.csv	问题 3 BMI 最优分组的各组信息
summary_improved.txt	问题 3 建模过程信息报告
best_model.pkl	问题 4 训练的最佳性能机器学习模型
best_model_info.json	问题 4 记录最佳模型的参数
analysis_report.txt	问题 4 机器学习模型训练报告
model_metrics.xlsx	问题 4 各模型多性能指标汇总
resample_stats.xlsx	问题 4 训练各模型的数据集特征汇总
processed_data.xlsx	问题 4 处理过程中对附件数据的加工

附录 B: 核心 python 代码

0.1 这是数据预处理所用核心代码，进行数据读取并标准化，计算质量阈值，软门控判定并对样本质量评分，为进一步分析做好准备。

```

1
2 ### 数据预处理_核心代码
3
4
5 # 软门控判定
6 def qc_soft_flags(row, thr):
7     flags = []
8     O = pd.to_numeric(row.get(COL_O), errors="coerce")
9     M = pd.to_numeric(row.get(COL_M), errors="coerce")

```

```

10 N = pd.to_numeric(row.get(COL_N), errors="coerce")
11 AA = pd.to_numeric(row.get(COL_AA), errors="coerce")
12 P = pd.to_numeric(row.get(COL_P), errors="coerce")
13 GC13 = pd.to_numeric(row.get(COL_GC13), errors='coerce')
14 GC18 = pd.to_numeric(row.get(COL_GC18), errors='coerce')
15 GC21 = pd.to_numeric(row.get(COL_GC21), errors='coerce')
16
17 # 比率/计数
18 if (not np.isfinite(M)) or (M < thr['M5']):
19     flags.append("low_align(<M5)")
20 if (not np.isfinite(N)) or (N > thr['N95']):
21     flags.append("high_dup(>N95)")
22 if (not np.isfinite(AA)) or (AA > thr['AA95']):
23     flags.append("high_filtered(>AA95)")
24 if (not np.isfinite(O)) or (O < thr['O5']):
25     flags.append("low_unique(<O5)")
26
27 # 整体 GC
28 if (not np.isfinite(P)) or (P < thr['GC_LOW']):
29     flags.append("gc_overall_low(<P5)")
30 if np.isfinite(P) and (P > thr['GC_HIGH']):
31     flags.append("gc_overall_high(>0.60)")
32
33 # ΔGC
34 if np.isfinite(GC13) and np.isfinite(GC18) and np.isfinite(GC21) and np.
isfinite(P):
35     for k, gck in [('13',GC13),('18',GC18),('21',GC21)]:
36         z = ((gck - P) - thr['mu_d'][k]) / (thr['sd_d'][k] + 1e-9)
37         if abs(z) > 3:
38             flags.append(f"deltaGC{k}_>|3sd|")
39             break
40     else:
41         flags.append("deltaGC_missing")
42
43     return flags
44
45 # 对被软门控样本计算 Q
46 def compute_Q_for_flagged(row, thr, return_parts=False):
47     """返回 Q (及可选的各组成成分) """
48     def clip01(x): return np.minimum(1.0, np.maximum(0.0, x))
49     O = pd.to_numeric(row.get(COL_O), errors="coerce")
50     M = pd.to_numeric(row.get(COL_M), errors="coerce")

```

```

51 N = pd.to_numeric(row.get(COL_N), errors="coerce")
52 AA = pd.to_numeric(row.get(COL_AA), errors="coerce")
53 P = pd.to_numeric(row.get(COL_P), errors="coerce")
54 GC13 = pd.to_numeric(row.get(COL_GC13), errors='coerce')
55 GC18 = pd.to_numeric(row.get(COL_GC18), errors='coerce')
56 GC21 = pd.to_numeric(row.get(COL_GC21), errors='coerce')
57
58 # 分位区间 (缺失兜底)
59 O5 = thr['O5']; O95 = O5 * 1.5 if np.isfinite(O5) else (0 if np.isfinite(0
) else 1.0)
60 M5 = thr['M5']; M95 = min(0.95, M5 + 0.15) if np.isfinite(M5) else 0.90
61 N5, N95 = 0.0, (thr['N95'] if np.isfinite(thr['N95']) else 0.04)
62 AA5, AA95 = 0.0, (thr['AA95'] if np.isfinite(thr['AA95']) else 0.04)
63 eps = 1e-9
64
65 f_reads = clip01((np.log10(max(0,1.0)) - np.log10(max(O5,1.0))) /
66                 (np.log10(max(O95, O5+1)) - np.log10(max(O5,1.0)) + eps))
67 if np.isfinite(0) and np.isfinite(O5) else 0.5
68 f_align = clip01((M - M5) / (M95 - M5 + eps)) if np.isfinite(M) and np.
isfinite(M5) else 0.5
69 f_dup = 1.0 - clip01((N - N5) / (N95 - N5 + eps)) if np.isfinite(N) and
np.isfinite(N95) else 0.5
70 f_filt = 1.0 - clip01((AA - AA5) / (AA95 - AA5 + eps)) if np.isfinite(AA)
and np.isfinite(AA95) else 0.5
71
72 # 整体 GC: 温和惩罚 (0.6~1.0)
73 f_gc_overall = 1.0
74 if np.isfinite(P):
75     if P < thr['GC_LOW']:
76         P1 = thr['P1'] if np.isfinite(thr['P1']) else max(thr['GC_LOW'] -
0.02, 0.0)
77         span = max(thr['GC_LOW'] - P1, 1e-3)
78         frac = (P - P1) / span # [0,1]
79         f_gc_overall = 0.6 + 0.4 * clip01(frac)
80     elif P > thr['GC_HIGH']:
81         P99 = thr['P99'] if np.isfinite(thr['P99']) else (thr['GC_HIGH'] +
0.02)
82         span = max(P99 - thr['GC_HIGH'], 1e-3)
83         frac = (P99 - P) / span
84         f_gc_overall = 0.6 + 0.4 * clip01(frac)
85     else:
86         f_gc_overall = 1.0

```

```

86
87 # ΔGC: 取 13/18/21 的最小值
88 f_delta = 1.0
89 if np.isfinite(GC13) and np.isfinite(GC18) and np.isfinite(GC21) and np.
isfinite(P):
90     f_list = []
91     for k, gck in [('13',GC13),('18',GC18),('21',GC21)]:
92         z = ((gck - P) - thr['mu_d'][k]) / (thr['sd_d'][k] + eps)
93         f_list.append(1.0 - min(1.0, abs(z)/3.0))
94     f_delta = min(f_list)
95
96 # 几何平均 (读段/比对加倍权重)
97 prod = (f_reads**2) * (f_align**2) * f_dup * f_filt * f_gc_overall *
f_delta
98 Q = float(max(prod ** (1/8), Q_FLOOR))
99
100 if return_parts:
101     return Q, dict(f_reads=f_reads, f_align=f_align, f_dup=f_dup, f_filt=
f_filt,
102                   f_gc_overall=f_gc_overall, f_delta=f_delta)
103 return Q
104
105 # 选“质量最好”的记录 (同孕妇同一天仅保留一条)
106 def pick_best_per_day(df):
107     # ΔGC 最大绝对偏差 (tie-breaker)
108     d13 = pd.to_numeric(df[COL_GC13], errors='coerce') - pd.to_numeric(df[
COL_P], errors='coerce')
109     d18 = pd.to_numeric(df[COL_GC18], errors='coerce') - pd.to_numeric(df[
COL_P], errors='coerce')
110     d21 = pd.to_numeric(df[COL_GC21], errors='coerce') - pd.to_numeric(df[
COL_P], errors='coerce')
111     df["_deltaGC_maxabs"] = pd.concat([d13.abs(), d18.abs(), d21.abs()], axis
=1).max(axis=1)

```

0.2 这是问题一相关性分析所用核心代码，基于聚类自助法计算了带权相关性考虑了数据的聚类，并评估相关性的显著性，提供 p 值。

```

1
2 import pandas as pd
3 import numpy as np
4 from scipy import stats
5 import matplotlib.pyplot as plt
6 import seaborn as sns

```

```

7 import warnings, time
8 from tqdm import tqdm
9 warnings.filterwarnings('ignore')
10 plt.rcParams['font.sans-serif'] = ['SimHei']
11 plt.rcParams['axes.unicode_minus'] = False
12
13
14 # 带权Spearman: 秩→加权Pearson
15 def weighted_spearmanr(x, y, w):
16     x = np.asarray(x, dtype=float)
17     y = np.asarray(y, dtype=float)
18     w = np.asarray(w, dtype=float)
19
20     mask = np.isfinite(x) & np.isfinite(y) & np.isfinite(w) & (w > 0)
21     if mask.sum() <= 2:
22         return np.nan
23
24     xr = stats.rankdata(x[mask], method='average')
25     yr = stats.rankdata(y[mask], method='average')
26     ww = w[mask]
27
28     ww_sum = ww.sum()
29     if ww_sum <= 0:
30         return np.nan
31
32     def wmean(arr, ww): return (ww * arr).sum() / ww_sum
33     xbar = wmean(xr, ww)
34     ybar = wmean(yr, ww)
35     xc = xr - xbar
36     yc = yr - ybar
37
38     cov_w = (ww * xc * yc).sum() / ww_sum
39     varx = (ww * xc * xc).sum() / ww_sum
40     vary = (ww * yc * yc).sum() / ww_sum
41     if varx <= 0 or vary <= 0:
42         return np.nan
43     return cov_w / np.sqrt(varx * vary)
44
45 # 聚类自助法+带权Spearman
46 def cluster_bootstrap_weighted_spearman(
47     data, col1, col2, cluster_col, weight_col,
48     n_bootstrap=1000, confidence_level=0.95, random_state=None

```

```

49 ):
50     valid = ~(data[col1].isna() | data[col2].isna())
51     if weight_col in data.columns:
52         w = pd.to_numeric(data[weight_col], errors='coerce').fillna(0.0)
53     else:
54         w = pd.Series(1.0, index=data.index)
55
56     original_corr = weighted_spearmanr(data.loc[valid, col1], data.loc[valid,
57 col2], w.loc[valid])
58
59     uniq = data[cluster_col].astype(str).unique()
60     n_clusters = len(uniq)
61     cluster_dict = {}
62     for c in uniq:
63         sub = data[data[cluster_col].astype(str) == str(c)][[col1, col2]].copy
64         ()
65         sub['__w__'] = w.loc[sub.index].values
66         cluster_dict[c] = sub.values # shape: (n_i, 3)
67
68     rng = np.random.default_rng(random_state)
69     boot_corrs = np.zeros(n_bootstrap, dtype=float)
70     valid_count = 0
71
72     for i in range(n_bootstrap):
73         # 有放回抽簇
74         picked = rng.choice(uniq, size=n_clusters, replace=True)
75         parts = [cluster_dict[c] for c in picked if c in cluster_dict]
76         if not parts:
77             continue
78         samp = np.vstack(parts) #[:,0]=col1,[:,1]=col2,[:,2]=w
79         x, y, ww = samp[:, 0], samp[:, 1], samp[:, 2]
80         corr = weighted_spearmanr(x, y, ww)
81         if np.isfinite(corr):
82             boot_corrs[valid_count] = corr
83             valid_count += 1
84
85     if valid_count == 0:
86         return dict(correlation=original_corr, p_value=np.nan, ci_lower=np.nan
87 ,
88                     ci_upper=np.nan, se=np.nan, n_bootstrap=0)
89
90     boot_corrs = boot_corrs[:valid_count]

```



```

88     se = float(np.std(boot_corrs))
89     alpha = 1 - confidence_level
90     ci_lower = float(np.percentile(boot_corrs, alpha/2 * 100))
91     ci_upper = float(np.percentile(boot_corrs, (1 - alpha/2) * 100))
92
93     # 简单双尾p值：看自助分布相对于0的尾部概率
94     if original_corr >= 0:
95         p_value = 2 * np.mean(boot_corrs <= 0)
96     else:
97         p_value = 2 * np.mean(boot_corrs >= 0)
98     p_value = float(min(1.0, max(0.0, p_value)))
99
100     return dict(
101         correlation=float(original_corr),
102         p_value=p_value,
103         ci_lower=ci_lower,
104         ci_upper=ci_upper,
105         se=se,
106         n_bootstrap=int(valid_count)
107     )
108
109
110 def main():
111     df, used_path, used_sheet, last_err= None, None, None, None
112     df = pd.read_excel("附件-男胎检测数据-处理.xlsx")
113
114     #权重列
115     weight_col = 'w_Q'
116     if weight_col not in df.columns:
117         print("警告：未找到 w_Q 列，默认权重=1")
118         df[weight_col] = 1.0
119     df[weight_col] = pd.to_numeric(df[weight_col], errors='coerce').fillna
120     (1.0).clip(lower=0)
121
122     columns_to_analyze = [
123         '年龄', '检测孕周', '孕妇BMI', '原始读段数', '在参考基因组上比对的比例', '
124         重复读段的比例',
125         '唯一比对的读段数', 'GC含量', '13号染色体的Z值', '18号染色体的Z值', '21号
126         染色体的Z值',
127         'X染色体的Z值', 'Y染色体的Z值', 'Y染色体浓度', 'X染色体浓度',
128         '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC含量', '被过滤
129         掉读段数的比例'

```

```

126 ]
127 exists = [c for c in columns_to_analyze if c in df.columns]
128 if len(exists) < len(columns_to_analyze):
129     missing = sorted(set(columns_to_analyze) - set(exists))
130     print(f"提示：以下列不存在，将自动忽略：{missing}")
131     columns_to_analyze = exists
132
133 #聚类列
134 cluster_col = '孕妇代码'
135 if cluster_col not in df.columns:
136     for cand in ['孕妇ID', '孕妇编号', 'ID', '编号', '代码']:
137         if cand in df.columns:
138             cluster_col = cand
139             break
140     else:
141         print("未找到聚类标识列，将使用索引创建虚拟簇")
142         df[cluster_col] = df.index
143
144 #只取需要的列 + 转数值
145 data = df[columns_to_analyze + [cluster_col, weight_col]].copy()
146 for c in columns_to_analyze:
147     data[c] = pd.to_numeric(data[c], errors='coerce')
148
149 uniq = data[cluster_col].nunique()
150 total = len(data)
151 print(f"独立孕妇数：{uniq}；总观察数：{total}；平均每孕妇检测次数：{total/
152 uniq:.2f}")
153 print("缺失值统计：")
154 print(data[columns_to_analyze].isnull().sum())
155
156 n_bootstrap = 1000
157 print(f"\n开始带权聚类自助法 (B={n_bootstrap}) ... \n")
158
159 n_vars = len(columns_to_analyze)
160 corrM = np.zeros((n_vars, n_vars)); pM = np.zeros_like(corrM)
161 loM = np.zeros_like(corrM); upM = np.zeros_like(corrM)
162 seM = np.zeros_like(corrM)
163
164 start = time.time()
165 tot_pairs = n_vars * (n_vars + 1) // 2
166 with tqdm(total=tot_pairs, desc="计算进度") as pbar:
    for i, c1 in enumerate(columns_to_analyze):

```

```

167         for j in range(i, n_vars):
168             c2 = columns_to_analyze[j]
169             if i == j:
170                 corrM[i,j]=1.0; pM[i,j]=0.0; loM[i,j]=1.0
171                 upM[i,j]=1.0; seM[i,j]=0.0
172             else:
173                 res = cluster_bootstrap_weighted_spearman(
174                     data, c1, c2, cluster_col, weight_col,
175                     n_bootstrap=n_bootstrap, confidence_level=0.95
176                 )
177                 corrM[i,j]=corrM[j,i]=res['correlation']
178                 pM[i,j]=pM[j,i]=res['p_value']
179                 loM[i,j]=loM[j,i]=res['ci_lower']
180                 upM[i,j]=upM[j,i]=res['ci_upper']
181                 seM[i,j]=seM[j,i]=res['se']
182             pbar.update(1)
183     dur = time.time() - start
184     print(f"\n完成! 用时 {dur:.1f}s")
185
186     #输出与可视化
187     spearman_df = pd.DataFrame(corrM, index=columns_to_analyze,
188                                columns=columns_to_analyze)
189     p_df = pd.DataFrame(pM, index=columns_to_analyze,
190                          columns=columns_to_analyze)
191     lo_df = pd.DataFrame(loM, index=columns_to_analyze,
192                           columns=columns_to_analyze)
193     up_df = pd.DataFrame(upM, index=columns_to_analyze,
194                           columns=columns_to_analyze)
195     se_df = pd.DataFrame(seM, index=columns_to_analyze,
196                           columns=columns_to_analyze)
197
198     sig_df = pd.DataFrame(index=columns_to_analyze, columns=columns_to_analyze
199 )
200     for i in range(n_vars):
201         for j in range(n_vars):
202             if i==j: sig_df.iloc[i,j]='-'; continue
203             pv=pM[i,j]
204             if np.isnan(pv): sig='NA'
205             elif pv<1e-3: sig='***'
206             elif pv<1e-2: sig='**'
207             elif pv<5e-2: sig='*'
208             else: sig='ns'

```

```

208         sig_df.iloc[i,j]=sig
209
210     #保存
211     with pd.ExcelWriter('聚类自助法_斯皮尔曼相关分析结果_带权.xlsx', engine='
openpyxl') as w:
212         info = pd.DataFrame({
213             '项目':['分析方法','是否使用权重',
214                 '权重列','独立孕妇数','总观察数',
215                 'Bootstrap次数','置信水平','计算用时(秒)',
216                 '数据文件','工作表'],
217             '值': ['Cluster Bootstrap + Weighted Spearman',
218                 '是','w_Q',uniq,total,n_bootstrap,'95%',
219                 f'{dur:.1f}','used_path,used_sheet']
220         })
221         info.to_excel(w, sheet_name='分析信息', index=False)
222         spearman_df.to_excel(w, sheet_name='相关系数矩阵')
223         p_df.to_excel(w,sheet_name='P值矩阵_聚类校正(带权)')
224         lo_df.to_excel(w,sheet_name='置信区间下限')
225         up_df.to_excel(w,sheet_name='置信区间上限')
226         se_df.to_excel(w,sheet_name='标准误')
227         sig_df.to_excel(w,sheet_name='显著性标记')
228         data[columns_to_analyze].describe().to_excel(w, sheet_name='描述统计(
未加权)')
229
230     #配对列表
231     rows=[]
232     for i in range(n_vars):
233         for j in range(i+1, n_vars):
234             rows.append({
235                 '指标1':columns_to_analyze[i],
236                 '指标2':columns_to_analyze[j],
237                 '相关系数':corrM[i,j],
238                 'P值(聚类校正,带权)':pM[i,j],
239                 '95%下限':loM[i,j],
240                 '95%上限':upM[i,j],
241                 '标准误':seM[i,j],
242                 '显著性':sig_df.iloc[i,j],
243                 '置信区间包含0': '是' if (np.isfinite(loM[i,j]) and loM[i,j]
<= 0 <= upM[i,j])
244                 else '否'
245             })
246     pairs = pd.DataFrame(rows).sort_values(['P值(聚类校正,带权)','相关系数'],

```

```

247         ascending=[True, False])
248 pairs.to_excel('聚类自助法_所有配对结果_带权.xlsx', index=False)
249 sig_pairs = pairs[pairs['P值(聚类校正,带权)']<0.05]
250 if not sig_pairs.empty:
251     sig_pairs.to_excel('聚类自助法_显著配对结果_带权.xlsx', index=False)
252
253 # 可视化
254 plt.figure(figsize=(18,16))
255 sns.heatmap(spearman_df, annot=True, fmt='.2f',
256             cmap='RdBu_r', center=0, vmin=-1,
257             vmax=1, square=True, linewidths=0.5,
258             cbar_kws={"shrink":0.8, "label": "带权Spearman相关"})
259 plt.title('带权Spearman相关系数热力图 (聚类自助法)',
260          fontsize=18, pad=20)
261 plt.xticks(rotation=45, ha='right'); plt.yticks(rotation=0)
262 plt.tight_layout()
263 plt.savefig('聚类自助法_相关系数热力图_带权.png', dpi=300)
264
265 plt.figure(figsize=(18,16))
266 sns.heatmap(p_df, annot=True, fmt='.3f', cmap='YlOrRd_r',
267             vmin=0, vmax=1, square=True, linewidths=0.5,
268             cbar_kws={"shrink":0.8, "label": "P值 (聚类校正,带权)"})
269 plt.title('P值热力图 (聚类自助法校正,带权)', fontsize=18, pad=20)
270 plt.xticks(rotation=45, ha='right'); plt.yticks(rotation=0)
271 plt.tight_layout(); plt.savefig('聚类自助法_P值热力图_带权.png', dpi=300)
272
273 # 显著性热力图
274 sig_numeric = np.zeros((n_vars, n_vars))
275 mapping = {'***':3, '**':2, '*':1, 'ns':0, '-':-1}
276 for i in range(n_vars):
277     for j in range(n_vars):
278         sig_numeric[i,j] = mapping.get(sig_df.iloc[i,j], np.nan)
279 plt.figure(figsize=(18,16))
280 cmap = sns.color_palette(['#d4d4d4', '#808080', '#ffcccc', '#ff6666', '#ff0000'],
281                          as_cmap=True)
282 sns.heatmap(sig_numeric, annot=sig_df.values, fmt='s', cmap=cmap, vmin=-1,
283             vmax=3,
284             square=True, linewidths=0.5, cbar_kws={"shrink":0.8, "label": "
显著性水平"})
285 cbar = plt.gcf().axes[-1]
286 cbar.set_yticklabels(['对角线', 'ns', '*', '**', '***'])

```

```

286     plt.title('显著性热力图 (聚类自助法校正, 带权)', fontsize=18, pad=20)
287     plt.xticks(rotation=45, ha='right'); plt.yticks(rotation=0)
288     plt.tight_layout(); plt.savefig('聚类自助法_显著性热力图_带权.png', dpi
    =300)
289
290 if __name__ == '__main__':
291     main()

```

0.3 这是广义加性混合模型 GAMM 求解核心程序, 主要目标是分析孕周、BMI 等因素与 Y 染色体浓度之间的关系。它对数据进行预处理, 处理缺失值, 应用加权分析, 并通过使用立方体回归样条和混合线性模型探索最佳的 GAMM 配置。

```

1
2 import re, warnings
3 import numpy as np
4 import pandas as pd
5 import statsmodels.api as sm
6 from scipy import stats
7 from patsy import dmatrix, build_design_matrices
8 import matplotlib, matplotlib.pyplot as plt
9 from pathlib import Path
10
11 matplotlib.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial
    Unicode MS", "DejaVu Sans"]
12 matplotlib.rcParams["axes.unicode_minus"] = False
13
14 CANDIDATES = [
15     ("附件-男胎检测数据-处理.xlsx", "附件-男胎检测数据-处理"),
16     ("附件.xlsx", "男胎检测数据"),
17 ]
18 COL_ID, COL_Y, COL_GA, COL_BMI, COL_AGE, COL_W = "孕妇代码", "Y染色体浓度", "
    检测孕周", "孕妇BMI", "年龄", "w_Q"
19
20 out_dir = Path("./model_outputs_gamm_w"); out_dir.mkdir(parents=True, exist_ok
    =True)
21
22 def parse_gestation_to_weeks(x):
23     if pd.isna(x): return np.nan
24     s = str(x).strip()
25     try: return float(s)
26     except: pass
27     m = re.search(r'(\d+)\s*(?:周|w|W)?\s*\+?\s*(\d+)\s*(?:天|d|D)?', s)
28     if m:

```

```

29         w = float(m.group(1)); d = float(m.group(2)) if m.group(2) else 0.0
30         return w + d/7.0
31     m2 = re.search(r'(\d+)', s)
32     return float(m2.group(1)) if m2 else np.nan
33
34 def smithson_verkuilen(y):
35     y = y.clip(0,1); n = y.notna().sum()
36     return (y*(n-1)+0.5)/n
37
38 def inv_logit(z): return 1/(1+np.exp(-z))
39 def AIC(llf, k): return 2*k - 2*llf
40 def BIC(llf, k, n): return np.log(n)*k - 2*llf
41
42 def wmean(x, w):
43     x = pd.to_numeric(x, errors="coerce"); w = pd.to_numeric(w, errors="coerce")
44     x = x.clip(lower=0)
45     m = np.nansum(w * x) / np.nansum(w) if np.nansum(w) > 0 else np.nan
46     return m
47
48 def fit_mixedlm_weighted(endog, exog, groups, weights, exog_re=None):
49     w = np.asarray(weights, dtype=float)
50     try:
51         md = sm.MixedLM(endog, exog, groups=groups, exog_re=exog_re, weights=w)
52         return md.fit(reml=False, method="lbfgs")
53     except (TypeError, ValueError):
54         print("当前 statsmodels MixedLM 不支持 weights")
55         sw = np.sqrt(np.clip(w, 1e-8, None))
56         endog_t = endog * sw
57         exog_t = exog * sw[:, None]
58         exre_t = exog_re * sw[:, None] if exog_re is not None else None
59         md = sm.MixedLM(endog_t, exog_t, groups=groups, exog_re=exre_t)
60         return md.fit(reml=False, method="lbfgs")
61
62 def lrt(res_small, res_large, df_diff=None):
63     LR = 2*(res_large.llf - res_small.llf)
64     if df_diff is None:
65         df_diff = len(res_large.params) - len(res_small.params)
66     p = stats.chi2.sf(LR, df_diff)
67     return LR, df_diff, p
68 #读取数据

```

```

69 last_err = None
70 for path, sheet in CANDIDATES:
71     try:
72         df = pd.read_excel(path, sheet_name=sheet)
73         used_path, used_sheet = path, sheet
74         break
75     except Exception as e:
76         last_err = e
77
78 for c in [COL_ID, COL_Y, COL_GA, COL_BMI]:
79     if c not in df.columns:
80         raise SystemExit(f"找不到列: {c} (数据源: {used_path}/{used_sheet}) ")
81 if COL_W not in df.columns:
82     print("[提示] 未找到 w_Q 列, 默认权重=1")
83     df[COL_W] = 1.0
84
85 use_cols = [COL_ID, COL_Y, COL_GA, COL_BMI, COL_W] + ([COL_AGE] if COL_AGE in
86     df.columns else [])
87 df = df[use_cols].copy()
88 df.rename(columns={COL_ID:"id", COL_Y:"Y", COL_GA:"GA_raw", COL_BMI:"BMI_raw",
89     COL_W:"w"}, inplace=True)
90 if COL_AGE in df.columns: df.rename(columns={COL_AGE:"Age_raw"}, inplace=True)
91
92 # 变量转换
93 df["GA"] = df["GA_raw"].apply(parse_gestation_to_weeks)
94 df["BMI"] = pd.to_numeric(df["BMI_raw"], errors="coerce")
95 if "Age_raw" in df.columns: df["Age"] = pd.to_numeric(df["Age_raw"], errors="
96     coerce")
97 df["w"] = pd.to_numeric(df["w"], errors="coerce").fillna(1.0).clip(lower=0)
98
99 df = df[["id","Y","GA","BMI","w"] + ([ "Age"] if "Age" in df.columns else [])].
100 dropna().copy()
101 print(f"样本量: {len(df)}; 孕妇个体数: {df['id'].nunique()}; 有效权重和={df['w
102     '].sum():.1f}")
103
104 # SV + logit
105 df["Y_star"] = smithson_verkuilen(df["Y"])
106 df["logitY"] = np.log(df["Y_star"]/(1-df["Y_star"]))
107
108 # 加权中心化
109 GA_mu = wmean(df["GA"], df["w"])
110 BMI_mu = wmean(df["BMI"], df["w"])

```



```

106 df["GA_c"] = df["GA"] - GA_mu
107 df["BMI_c"] = df["BMI"] - BMI_mu
108 use_age = "Age" in df.columns
109 if use_age:
110     Age_mu = wmean(df["Age"], df["w"])
111     df["Age_c"] = df["Age"] - Age_mu
112
113 # 线性 LMM
114 X_lin = pd.DataFrame({"Intercept":1.0, "GA_c":df["GA_c"], "BMI_c":df["BMI_c"]})
115 if use_age: X_lin["Age_c"] = df["Age_c"]
116 res_lin = fit_mixedlm_weighted(df["logitY"].values, X_lin.values, df["id"], df["w"].values)
117 k_lin = len(res_lin.params)
118 aic_lin = AIC(res_lin.llf, k_lin); bic_lin = BIC(res_lin.llf, k_lin, len(df))
119 print(f"[线性LMM·带权] logLik={res_lin.llf:.3f} AIC={aic_lin:.2f} BIC={bic_lin:.2f}")
120
121 # GAMM: cr样条+随机截距
122 best = None
123 df_grid = [4,5,6]
124 for df_ga in df_grid:
125     for df_b in df_grid:
126         formula = f"1 + cr(GA_c, df={df_ga}) + cr(BMI_c, df={df_b})" + (" + Age_c" if use_age else "")
127         try:
128             X_try = dmatrix(formula, df, return_type="dataframe")
129             res = fit_mixedlm_weighted(df["logitY"].values, X_try.values, df["id"], df["w"].values)
130             k = len(res.params)
131             aic = AIC(res.llf, k); bic = BIC(res.llf, k, len(df))
132             print(f" [GAMM·带权] df_GA={df_ga}, df_BMI={df_b} logLik={res.llf:.3f} AIC={aic:.2f}")
133             if (best is None) or (aic < best["aic"]):
134                 best = dict(res=res, aic=aic, bic=bic, k=k, df_ga=df_ga, df_bmi=df_b,
135                             formula=formula, design_info=X_try.design_info)
136         except Exception as e:
137             print(f" [跳过] df_GA={df_ga}, df_BMI={df_b} 收敛失败: {e}")
138
139 # 退路: 孕周样条+BMI 线性
140 fallback = False

```

```

141 if best is None:
142     fallback = True
143     for df_ga in df_grid:
144         formula = f"1 + cr(GA_c, df={df_ga}) + BMI_c" + (" + Age_c" if use_age
145             else "")
146         try:
147             X_try = dmatrix(formula, df, return_type="dataframe")
148             res = fit_mixedlm_weighted(df["logitY"].values, X_try.values, df
149                 ["id"], df["w"].values)
150             k = len(res.params)
151             aic = AIC(res.llf, k); bic = BIC(res.llf, k, len(df))
152             print(f" [GAMM-lite·带权] df_GA={df_ga} logLik={res.llf:.3f}
153                 AIC={aic:.2f}")
154             if (best is None) or (aic < best["aic"]):
155                 best = dict(res=res, aic=aic, bic=bic, k=k, df_ga=df_ga,
156                     df_bmi=None,
157                     formula=formula, design_info=X_try.design_info)
158             except Exception as e:
159                 print(f" [跳过] GAMM-lite df_GA={df_ga} 收敛失败: {e}")
160
161 if best is None:
162     raise SystemExit("所有 GAMM 设定均收敛失败, 请检查数据或降低样条自由度。")
163
164 res_gamm = best["res"]; design_info = best["design_info"]
165 print("\n[GAMM 最优·带权] 公式:", best["formula"])
166 print(f"[GAMM 最优·带权] logLik={res_gamm.llf:.3f} AIC={best['aic']:.2f}")
167
168 # LRT
169 df_diff = best["k"] - k_lin
170 LR, dfd, pval = lrt(res_lin, res_gamm, df_diff=df_diff)
171 print(f"\n[非线性整体检验·带权] 线性LMM → GAMM: LR={LR:.3f}, df={dfd}, p={pval
172     :.4g}")
173
174 # 固定效应表
175 X_train = dmatrix(best["formula"], df, return_type="dataframe")
176 p_fix = X_train.shape[1]
177 coef = pd.Series(res_gamm.params[:p_fix], index=[f"FE_{i}" for i in range(
178     p_fix)], name="Estimate")
179 se = pd.Series(res_gamm.bse[:p_fix], index=coef.index, name="SE")
180 z = pd.Series(coef.values/se.values, index=coef.index, name="z")
181 p = pd.Series(2*stats.norm.sf(np.abs(z.values)), index=coef.index, name="p
182 ")

```

```

176 ciL = pd.Series(coef.values - stats.norm.ppf(0.975)*se.values, index=coef.
      index, name="CI_low")
177 ciH = pd.Series(coef.values + stats.norm.ppf(0.975)*se.values, index=coef.
      index, name="CI_high")
178 pd.concat([coef,se,z,p,ciL,ciH], axis=1).to_excel(out_dir/"
      GAMM_coef_table_weighted.xlsx")
179
180 with open(out_dir/"GAMM_model_summary_weighted.txt","w",encoding="utf-8") as f
      :
181     f.write(
182         f"数据源: {used_path}/{used_sheet}\n"
183         f"样本量: {len(df)}; 孕妇数: {df['id'].nunique()}; 权重和: {df['w'].
sum():.1f}\n"
184         f"线性LMM·带权: logLik={res_lin.llf:.3f}, AIC={aic_lin:.2f}\n"
185         f"GAMM最优·带权: {best['formula']}\n"
186         f"logLik={res_gamm.llf:.3f}, AIC={best['aic']:.2f}\n"
187         f"LRT(线性→GAMM): LR={LR:.3f}, df={dfd}, p={pval:.4g}\n"
188         f"说明: {'退路模型 (孕周样条+BMI线性)' if fallback else '二维样条 (孕
周+BMI)'}\n"
189     )
190
191 #部分效应图
192 def partial_plot(var_name, x_grid, GA_med, BMI_med, Age_med=None, title="", fn
      ="plot.png"):
193     new = pd.DataFrame({
194         "GA_c": np.repeat(GA_med, len(x_grid)),
195         "BMI_c": np.repeat(BMI_med, len(x_grid))
196     })
197     new[var_name] = x_grid
198     if use_age: new["Age_c"] = Age_med
199     X_new = build_design_matrices([design_info], new)[0]
200     X_new = np.asarray(X_new)
201     beta = res_gamm.params[:p_fix]
202     cov = res_gamm.cov_params()[:p_fix, :p_fix]
203     lin = X_new @ beta
204     var_lin = np.einsum("ij,jk,ik->i", X_new, cov, X_new)
205     se_lin = np.sqrt(np.maximum(var_lin, 0))
206     zcrit = stats.norm.ppf(0.975)
207     lo_prob = inv_logit(lin - zcrit*se_lin)
208     hi_prob = inv_logit(lin + zcrit*se_lin)
209     y_prob = inv_logit(lin)
210     plt.figure(figsize=(7,5))

```

```

211     if var_name == "GA_c":
212         x_axis = x_grid + wmean(df["GA"], df["w"])
213         plt.xlabel("检测孕周 (周) ")
214     else:
215         x_axis = x_grid + wmean(df["BMI"], df["w"])
216         plt.xlabel("BMI")
217     plt.plot(x_axis, y_prob, label="预测均值")
218     plt.fill_between(x_axis, lo_prob, hi_prob, alpha=0.25, label="95%CI")
219     plt.ylabel("预测Y浓度 (比例) ")
220     plt.title(title)
221     plt.legend(); plt.tight_layout(); plt.savefig(out_dir/fn, dpi=220); plt.
close()
222
223 # 用带权“中位数”和范围
224 def wmedian(x, w):
225     x = pd.to_numeric(x, errors="coerce"); w = pd.to_numeric(w, errors="coerce
").clip(lower=0)
226     idx = np.argsort(x.values); xv, wv = x.values[idx], w.values[idx]
227     cw = np.cumsum(wv); total = cw[-1] if len(cw)>0 else np.nan
228     if not np.isfinite(total) or total<=0: return np.nan
229     return xv[np.searchsorted(cw, 0.5*total)]
230
231 GA_med = wmedian(df["GA_c"], df["w"]); BMI_med = wmedian(df["BMI_c"], df["w
"]); Age_med = wmedian(df["Age_c"], df["w"]) if use_age else None
232 GA_lo, GA_hi = df["GA_c"].quantile(0.02), df["GA_c"].quantile(0.98)
233 BMI_lo, BMI_hi = df["BMI_c"].quantile(0.02), df["BMI_c"].quantile(0.98)
234 GA_grid = np.linspace(GA_lo, GA_hi, 150)
235 BMI_grid = np.linspace(BMI_lo, BMI_hi, 150)
236
237 partial_plot("GA_c", GA_grid, GA_med, BMI_med, Age_med,
238             title="GAMM (带权/兼容) 部分效应: 孕周", fn="GAMM_w_partial_GA.
png")
239 partial_plot("BMI_c", BMI_grid, GA_med, BMI_med, Age_med,
240             title="GAMM (带权/兼容) 部分效应: BMI", fn="GAMM_w_partial_BMI.
png")
241
242 print("\n[GAMM·带权] 输出: ", out_dir.resolve())

```

0.4 这是使用线性混合模型 (LMM) 分析同一孕期数据的核心程序。它计算加权均值, 并使用混合效应方法拟合变换后的数据 (logit 转换)。输出包括模型评估所需的 AIC、BIC 和对数似然值。用于与 GAMM 模型对比。

```

2 import re
3 import numpy as np
4 import pandas as pd
5 import statsmodels.api as sm
6 import statsmodels.formula.api as smf
7 from patsy import dmatrices
8 from scipy import stats
9 import matplotlib, matplotlib.pyplot as plt
10 from pathlib import Path
11 import warnings
12
13
14 matplotlib.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial
    Unicode MS", "DejaVu Sans"]
15 matplotlib.rcParams["axes.unicode_minus"] = False
16
17 CANDIDATES = [
18     ("附件-男胎检测数据-处理.xlsx", "附件-男胎检测数据-处理"),
19     ("附件.xlsx", "男胎检测数据"),
20 ]
21 COL_ID, COL_Y, COL_GA, COL_BMI, COL_AGE, COL_W = "孕妇代码", "Y染色体浓度", "
    检测孕周", "孕妇BMI", "年龄", "w_Q"
22 out_dir = Path("./model_outputs_lmm_w"); out_dir.mkdir(parents=True, exist_ok=
    True)
23
24 def parse_gestation_to_weeks(x):
25     if pd.isna(x): return np.nan
26     s = str(x).strip()
27     try: return float(s)
28     except: pass
29     m = re.search(r'(\d+)\s*(?:周|w|W)?\s*\+?\s*(\d+)\s*(?:天|d|D)?', s)
30     if m:
31         w = float(m.group(1)); d = float(m.group(2)) if m.group(2) else 0.0
32         return w + d/7.0
33     m2 = re.search(r'(\d+)', s)
34     return float(m2.group(1)) if m2 else np.nan
35
36 def smithson_verkuilen(y):
37     y = y.clip(0, 1); n = y.notna().sum()
38     return (y*(n-1) + 0.5) / n
39
40 def wmean(x, w):

```

```

41     x = pd.to_numeric(x, errors="coerce"); w = pd.to_numeric(w, errors="coerce")
42     return np.nansum(w*x)/np.nansum(w) if np.nansum(w)>0 else np.nan
43
44 def mixedlm_lrt(res_small, res_large, df_diff=None):
45     LR = 2.0*(res_large.llf - res_small.llf)
46     if df_diff is None:
47         df_diff = len(res_large.params) - len(res_small.params)
48     p = stats.chi2.sf(LR, df_diff)
49     return LR, df_diff, p
50
51 # 读取
52 last_err = None
53 for path, sheet in CANDIDATES:
54     try:
55         df = pd.read_excel(path, sheet_name=sheet)
56         used_path, used_sheet = path, sheet
57         break
58     except Exception as e:
59         last_err = e
60 else:
61     raise SystemExit(f"读取失败: {last_err}")
62
63 for c in [COL_ID, COL_Y, COL_GA, COL_BMI]:
64     if c not in df.columns:
65         raise SystemExit(f"找不到列: {c} (数据源: {used_path}/{used_sheet}) ")
66 if COL_W not in df.columns:
67     print("[提示] 未找到 w_Q 列, 默认权重=1")
68     df[COL_W] = 1.0
69
70 use_cols = [COL_ID, COL_Y, COL_GA, COL_BMI, COL_W] + ([COL_AGE] if COL_AGE in
71     df.columns else [])
72 df = df[use_cols].copy()
73 df = df.rename(columns={COL_ID:"id", COL_Y:"Y", COL_GA:"GA_raw", COL_BMI:"
74     BMI_raw", COL_W:"w"})
75 if COL_AGE in df.columns: df = df.rename(columns={COL_AGE:"Age_raw"})
76
77 df["GA"] = df["GA_raw"].apply(parse_gestation_to_weeks)
78 df["BMI"] = pd.to_numeric(df["BMI_raw"], errors="coerce")
79 if "Age_raw" in df.columns: df["Age"] = pd.to_numeric(df["Age_raw"], errors="
80     coerce")
81 df["w"] = pd.to_numeric(df["w"], errors="coerce").fillna(1.0).clip(lower=0)

```

```

79
80 keep = ["id", "Y", "GA", "BMI", "w"] + (["Age"] if "Age" in df.columns else [])
81 df = df[keep].dropna().copy()
82
83 # SV + logit
84 df["Y_star"] = smithson_verkuilen(df["Y"])
85 df["logitY"] = np.log(df["Y_star"]/(1-df["Y_star"]))
86
87 # 加权中心化
88 GA_mu = wmean(df["GA"], df["w"])
89 BMI_mu = wmean(df["BMI"], df["w"])
90 df["GA_c"] = df["GA"] - GA_mu
91 df["BMI_c"] = df["BMI"] - BMI_mu
92 use_age = "Age" in df.columns
93 if use_age:
94     Age_mu = wmean(df["Age"], df["w"])
95     df["Age_c"] = df["Age"] - Age_mu
96
97 print(f"样本量: {len(df)}; 孕妇个体数: {df['id'].nunique()}; 有效权重和={df['w']
    '].sum():.1f}")
98
99 # 带权拟合
100 def fit_mlm(formula_str):
101     w = np.asarray(df["w"].values, dtype=float)
102     try:
103         md = smf.mixedlm(formula_str, data=df, groups=df["id"], re_formula
            ="1", weights=w)
104         res = md.fit(reml=False, method="lbfgs")
105         # 保持统一接口
106         try:
107             res._exog_names = list(res.model.exog_names)
108         except Exception:
109             pass
110         return res
111     except (TypeError, ValueError):
112         print(f"[警告] MixedLM 不支持 weights, 公式 {formula_str} 改用  $\sqrt{w}$  伪加
            权。")
113         # 用 patsy 生成设计矩阵 (含截距, 并能拿到列名)
114         y_mat, X_mat = dmatrices(formula_str, df, return_type="dataframe")
115         y = np.asarray(y_mat).ravel()
116         X = np.asarray(X_mat); exog_names = list(X_mat.columns)
117         sw = np.sqrt(np.clip(w, 1e-8, None))

```

```

118     y_t = y * sw
119     X_t = X * sw[:, None]
120     Z_t = np.ones((len(df), 1)) * sw[:, None]    # 随机截距也乘  $\sqrt{w}$ 
121     md = sm.MixedLM(y_t, X_t, groups=df["id"], exog_re=Z_t)
122     res = md.fit(reml=False, method="lbfgs")
123     # 把名字挂到结果上
124     res._exog_names = exog_names
125     return res
126
127 fit0 = fit_mlm("logitY ~ 1")
128 fit1 = fit_mlm("logitY ~ 1 + GA_c")
129 fit2 = fit_mlm("logitY ~ 1 + GA_c + BMI_c")
130 if use_age:
131     fit3 = fit_mlm("logitY ~ 1 + GA_c + BMI_c + Age_c")
132     final_fit, final_name = fit3, "M3: GA + BMI + Age (带权/兼容) "
133 else:
134     final_fit, final_name = fit2, "M2: GA + BMI (带权/兼容) "
135
136 print("\n=== 最终模型摘要 ({}) ===".format(final_name))
137 print(final_fit.summary())
138
139 # LRT
140 LR_10 = mixedlm_lrt(fit0, fit1, df_diff=1)
141 LR_21 = mixedlm_lrt(fit1, fit2, df_diff=1)
142 print("\n=== 似然比检验 (带权/兼容) ===")
143 print(f"M0→M1 (+孕周) : LR={LR_10[0]:.3f}, df={LR_10[1]}, p={LR_10[2]:.4g}")
144 print(f"M1→M2 (+BMI) : LR={LR_21[0]:.3f}, df={LR_21[1]}, p={LR_21[2]:.4g}")
145 if use_age:
146     LR_32 = mixedlm_lrt(fit2, fit3, df_diff=1)
147     print(f"M2→M3 (+年龄) : LR={LR_32[0]:.3f}, df={LR_32[1]}, p={LR_32[2]:.4g}")
148
149 # 边际/条件 $R^2$  (简化近似)
150 try:
151     var_random = float(getattr(final_fit, "cov_re", np.array([[np.nan]])).iloc[0,0])
152 except Exception:
153     var_random = float(np.asarray(getattr(final_fit, "cov_re", np.array([[np.nan]])))[0,0])
154 var_resid = float(final_fit.scale)
155 var_fitted = float(np.var(final_fit.fittedvalues, ddof=1))
156 denom = var_fitted + var_resid if np.isfinite(var_fitted + var_resid) else np.

```



```

    nan
157 r2_c = var_fitted/denom if denom and denom>0 else np.nan
158 r2_m = max(0.0, r2_c - var_random/(var_random + var_resid + 1e-9)) if np.
    isfinite(r2_c) else np.nan
159 print("\n=== R2 (Nakagawa & Schielzeth, 带权/兼容) ===")
160 print(f"边际R2: {r2_m:.4f}; 条件R2: {r2_c:.4f}")
161 print("方差组成 (近似) : ", {'var_random': var_random, 'var_resid': var_resid
    })
162
163 # 等效孕周 & OR
164 # 固定效应的名字与向量
165 fe_names = getattr(final_fit, "_exog_names", None)
166 if fe_names is None and hasattr(final_fit, "model") and hasattr(final_fit.
    model, "exog_names"):
167     fe_names = list(final_fit.model.exog_names)
168
169 k_fe = int(getattr(final_fit, "k_fe", len(fe_names) if fe_names is not None
    else 0))
170 beta_all = np.asarray(final_fit.params)
171 beta_fe = beta_all[:k_fe]
172 cov_all = np.asarray(final_fit.cov_params())
173 cov_fe = cov_all[:k_fe, :k_fe]
174
175 if fe_names is not None and "GA_c" in fe_names and "BMI_c" in fe_names:
176     idx_ga = fe_names.index("GA_c")
177     idx_bmi = fe_names.index("BMI_c")
178     b_ga = float(beta_fe[idx_ga])
179     b_bmi = float(beta_fe[idx_bmi])
180     cov2 = cov_fe[[idx_ga, idx_bmi]][:, [idx_ga, idx_bmi]]
181
182     theta = -b_bmi / b_ga
183     d_b1 = b_bmi / (b_ga**2); d_b2 = -1.0 / b_ga
184     grad = np.array([[d_b1, d_b2]])
185     var_theta = float(np.squeeze(grad @ cov2 @ grad.T))
186     se = np.sqrt(max(var_theta, 0))
187     z = stats.norm.ppf(0.975); lo, hi = theta - z*se, theta + z*se
188
189     print("\n=== ‘等效孕周’ (带权/兼容) ===")
190     print(f"点估计: {theta:.3f} 周; 95%CI [{lo:.3f}, {hi:.3f}]; SE={se:.3f}")
191     print("\n=== 胜算比 OR (logit 尺度) ===")
192     print(f"孕周 +1 周: OR={np.exp(b_ga):.3f}")
193     print(f"BMI +1 : OR={np.exp(b_bmi):.3f}")

```

```

194 else:
195     print("\n[提示] 无法定位 GA_c/BMI_c 的系数（因列名丢失），跳过‘等效孕周/
      OR’。")
196
197 # 诊断与部分效应
198 def inv_logit(z): return 1.0/(1.0+np.exp(-z))
199 plt.figure(figsize=(7,5))
200 plt.scatter(final_fit.fittedvalues, final_fit.resid, s=10, alpha=0.6)
201 plt.axhline(0, color="k", lw=1); plt.xlabel("拟合值 (logitY)"); plt.ylabel("
      残差")
202 plt.title("残差-拟合图 (LMM·带权/兼容)")
203 plt.tight_layout(); plt.savefig(out_dir/"resid_vs_fitted_w.png", dpi=220)
204
205 # 用 names 来构造预测矩阵
206 def predict_curve(var_name, grid):
207     # 仅用固定效应名/参数预测
208     if fe_names is None:
209         return None, None
210     base = {nm:0.0 for nm in fe_names}
211     if "Intercept" in base: base["Intercept"] = 1.0
212
213     X_rows = []
214     for val in grid:
215         row = base.copy()
216         if var_name in row:
217             row[var_name] = val
218         X_rows.append([row[nm] for nm in fe_names])
219     X = np.asarray(X_rows)
220
221     yhat = inv_logit(X @ beta_fe) # 用固定效应参数
222     # 反中心化回原尺度
223     if var_name == "GA_c":
224         x_axis = grid + GA_mu
225     elif var_name == "BMI_c":
226         x_axis = grid + BMI_mu
227     else:
228         x_axis = grid
229     return x_axis, yhat
230
231
232 GA_grid = np.linspace(df["GA_c"].quantile(0.02), df["GA_c"].quantile(0.98),
      120)

```

```

233 BMI_grid = np.linspace(df["BMI_c"].quantile(0.02), df["BMI_c"].quantile(0.98),
    120)
234
235 xg, yg = predict_curve("GA_c", GA_grid)
236 if xg is not None:
237     plt.figure(figsize=(7,5))
238     plt.plot(xg, yg); plt.xlabel("检测孕周 (周)"); plt.ylabel("预测Y浓度 (比例)")
239     plt.title("部分效应: 孕周 (带权/兼容)")
240     plt.tight_layout(); plt.savefig(out_dir/"partial_effect_GA_w.png", dpi
    =220)
241
242 xb, yb = predict_curve("BMI_c", BMI_grid)
243 if xb is not None:
244     plt.figure(figsize=(7,5))
245     plt.plot(xb, yb); plt.xlabel("BMI"); plt.ylabel("预测Y浓度 (比例)")
246     plt.title("部分效应: BMI (带权/兼容)")
247     plt.tight_layout(); plt.savefig(out_dir/"partial_effect_BMI_w.png", dpi
    =220)
248 plt.close("all")
249
250 if fe_names is None:
251     fe_names = [f"x{i+1}" for i in range(k_fe)]
252 fe_se = np.asarray(final_fit.bse)[:k_fe]
253
254 coef = pd.DataFrame({"name": fe_names, "Estimate": beta_fe})
255 z     = coef["Estimate"].values / fe_se
256 p     = 2*stats.norm.sf(np.abs(z))
257 ciL   = coef["Estimate"].values - stats.norm.ppf(0.975)*fe_se
258 ciH   = coef["Estimate"].values + stats.norm.ppf(0.975)*fe_se
259
260 out_coef = pd.DataFrame({
261     "Estimate": coef["Estimate"].values,
262     "SE": fe_se,
263     "z": z,
264     "p": p,
265     "CI_low": ciL,
266     "CI_high": ciH
267 }, index=fe_names)
268 out_coef.to_excel(out_dir/"LMM_coef_table_weighted_compat.xlsx")
269
270

```

```

271 with open(out_dir/"model_summary_weighted_compat.txt","w",encoding="utf-8") as
    f:
272     f.write(
273         f"数据源: {used_path}/{used_sheet}\n"
274         f"样本量: {len(df)}; 孕妇数: {df['id'].nunique()}; 权重和: {df['w'].
sum():.1f}\n"
275         f"最终模型: {final_name}\n"
276         f"边际R2: {r2_m:.4f}; 条件R2: {r2_c:.4f}\n"
277         f"随机截距方差: {var_random:.6f}; 残差方差: {var_resid:.6f}\n"
278     )
279
280 print("\n[LMM·带权/兼容] 输出: ", out_dir.resolve())

```

0.5 这是解决问题二所用核心代码，进行数据预处理、阈值计算和对各种基因标记进行软质量控制标记，处理与 Y 染色体浓度及其他测量相关的数据，应用质量控制的阈值。

```

1
2 ### 问题2_核心代码
3
4
5 #单人 Ti 构造
6 def construct_Ti(df, res, design_info, work_for_centering):
7     GA_mean = work_for_centering["GA"].mean()
8     BMI_mean = work_for_centering["BMI"].mean()
9     Age_mean = work_for_centering["Age"].mean() if "Age" in work_for_centering
.columns else None
10
11     rows = []
12     for pid, g in df.sort_values([COL_ID, COL_GA]).groupby(COL_ID):
13         g = g.sort_values(COL_GA)
14         ga = g[COL_GA].to_numpy(float)
15         yy = g[COL_Y].to_numpy(float)
16         bmi0 = float(g[COL_BMI].iloc[0])
17         age0 = float(g[COL_AGE].iloc[0]) if (COL_AGE in g.columns) else None
18
19         thr = 0.04
20         cross_T = None
21         for j in range(1, len(ga)):
22             if (yy[j-1] < thr) and (yy[j] >= thr):
23                 t1, t2 = ga[j-1], ga[j]
24                 y1, y2 = yy[j-1], yy[j]
25                 w = 0.5 if (y2==y1) else np.clip((thr - y1)/(y2 - y1), 0.0,
1.0)

```

```

26         cross_T = t1 + w*(t2 - t1)
27         break
28
29     if cross_T is not None:
30         rows.append(dict(id=pid, BMI0=bmi0, Age0=age0, T=float(cross_T),
31                        source="interp", n_tests=len(g), first_ga=float(
32                        ga[0]),
33                        last_ga=float(ga[-1])))
34         continue
35
36     #无跨越 → 用带权 GAMM 的固定效应解根
37     def f(t):
38         lin = predict_logit_mean(design_info, res,
39                                  GA=np.array([t]),
40                                  BMI=np.array([bmi0]),
41                                  Age=np.array([age0]) if (age0 is not None
42                                  ) else None,
43                                  GA_mean=GA_mean, BMI_mean=BMI_mean,
44                                  Age_mean=Age_mean)
45         return inv_logit(lin) - thr
46
47     t_lo, t_hi = 8.0, 32.0
48     f_lo, f_hi = f(t_lo), f(t_hi)
49     if f_lo >= 0:
50         Ti = max(10.0, float(ga[0])) if len(ga)>0 else 10.0
51     elif f_hi <= 0:
52         Ti = max(32.0, float(ga[-1])) if len(ga)>0 else 32.0
53     else:
54         Ti = float(brentq(f, t_lo, t_hi, maxiter=200))
55     rows.append(dict(id=pid, BMI0=bmi0, Age0=age0, T=Ti,
56                      source="gamm", n_tests=len(g), first_ga=float(ga[0]),
57                      last_ga=float(ga[-1])))
58 return pd.DataFrame(rows).sort_values("BMI0").reset_index(drop=True)
59
60 #DP 分段
61 def precompute_sums_logT(Ti_df, weights=None):
62     z = np.log(np.clip(Ti_df["T"].values, 1e-6, None))
63     if weights is None:
64         w = np.ones_like(z)
65     else:
66         w = np.asarray(weights, float)

```

```

64     W    = np.concatenate([[0.0], np.cumsum(w)])
65     S    = np.concatenate([[0.0], np.cumsum(w * z)])
66     S2   = np.concatenate([[0.0], np.cumsum(w * z * z)])
67     return z, w, W, S, S2
68
69
70 #加权核ECDF
71 ALPHA_KERNEL = ALPHA_KERNEL
72 def kernel_ecdf_factory_weighted(samples, weights=None, alpha=ALPHA_KERNEL):
73     xs = np.asarray(samples, float)
74     n  = len(xs)
75     if n == 0: return lambda t: 0.0
76     if weights is None:
77         w = np.ones(n, float)
78     else:
79         w = np.asarray(weights, float).clip(min=0.0)
80         if w.sum() <= 0: w = np.ones(n, float)
81     mu = np.sum(w*xs)/np.sum(w)
82     var = np.sum(w*(xs-mu)**2)/np.sum(w)
83     sd = np.sqrt(max(var, 1e-12))
84     h_silverman = 1.06 * sd * (n ** (-1/5))
85     h = max(H_MIN, alpha * h_silverman)
86     w_norm = w / np.sum(w); inv_h = 1.0 / h
87     def Fhat(t):
88         t_arr = np.atleast_1d(t).astype(float)
89         Z = (t_arr[:, None] - xs[None, :]) * inv_h
90         vals = stats.norm.cdf(Z) @ w_norm
91         return float(vals[0]) if np.isscalar(t) else vals
92     return Fhat
93
94 #风险函数
95 def expected_risk_enhanced(F, t0, delta=DELTA, group_stats=None,
96                             group_idx=None, total_groups=None):
97     t_max = max(40.0, 28.0 + 6.0)
98     risk = LAMBDA_FAIL * (1.0 - F(t0))
99     if group_stats and group_idx is not None and total_groups is not None:
100         mean_T = group_stats['mean_T']
101         std_T = group_stats['std_T']
102         mean_BMI = group_stats['mean_BMI']
103         if mean_BMI > 35:
104             early_penalty, late_penalty = 0.5, 2.0
105         elif mean_BMI < 28:

```

```

106         early_penalty, late_penalty = 1.5, 0.8
107     else:
108         early_penalty, late_penalty = 1.0, 1.0
109     risk *= early_penalty
110     variability_factor = 1.0 + 0.1 * (std_T / max(mean_T, 1))
111     t_prev = 0.0; tk = t0
112     while tk <= t_max:
113         mass = max(F(tk) - F(t_prev), 0.0)
114         base_weight = stage_weight(tk)
115         if tk > 20: base_weight *= late_penalty
116         risk += base_weight * variability_factor * mass
117         t_prev = tk; tk += delta
118     tail = max(1.0 - F(t_prev), 0.0)
119     if tail > 0:
120         risk += stage_weight(t_max + 1e-6) * late_penalty *
variability_factor * tail
121     else:
122         t_prev = 0.0; tk = t0
123         while tk <= t_max:
124             mass = max(F(tk) - F(t_prev), 0.0)
125             risk += stage_weight(tk) * mass
126             t_prev = tk; tk += delta
127         tail = max(1.0 - F(t_prev), 0.0)
128         if tail > 0:
129             risk += stage_weight(t_max + 1e-6) * tail
130     return float(risk)
131
132
133 #最佳时点
134 def optimize_t_star_enhanced(F, group_stats, group_idx, total_groups):
135     mean_T = group_stats['mean_T']; std_T = group_stats['std_T']
136     median_T = group_stats['median_T']; mean_BMI = group_stats['mean_BMI']
137     if mean_BMI < 28:
138         win_center = max(WIN_MIN, min(median_T - std_T, 14))
139         win_width = min(6, 1.5 * std_T)
140     elif mean_BMI > 35:
141         win_center = min(WIN_MAX - 2, max(median_T, 16))
142         win_width = min(8, 2 * std_T)
143     else:
144         win_center = max(WIN_MIN + 2, min(median_T - 0.3 * std_T, WIN_MAX - 3)
145     )
146     win_width = min(7, 1.8 * std_T)

```

```

146 search_min = max(WIN_MIN, win_center - win_width/2)
147 search_max = min(WIN_MAX, win_center + win_width/2)
148 if search_max - search_min < 4:
149     if mean_BMI < 30: search_max = min(WIN_MAX, search_min + 4)
150     else: search_min = max(WIN_MIN, search_max - 4)
151 t_grid = np.arange(search_min, search_max + 1e-9, T_GRID_STEP)
152 risks = [expected_risk_enhanced(F, t, group_stats=group_stats,
153     group_idx=group_idx, total_groups=total_groups) for t in t_grid]
154 if len(risks) == 0: return float(mean_T), 1.0
155 k = int(np.argmin(risks))
156 return float(t_grid[k]), float(risks[k])

```

0.6 这是解决问题三所用核心代码，继续运用了问题一的广义加性模型和正交化技术分析孕期相关数据，处理 BMI、年龄、身高和体重等特征，预测 Y 染色体浓度。

```

1
2 ### 问题3_核心代码
3
4
5
6 # 拟合 GAMM
7 def fit_gamm(df):
8     def sv(y):
9         y = np.clip(y, 0, 1); n = np.isfinite(y).sum()
10        return (y*(n-1)+0.5)/max(n,1)
11
12    work = df.copy()
13    work["GA"] = work[COL_GA]
14    work["BMI"] = work[COL_BMI]
15    if COL_AGE in work.columns: work["Age"] = work[COL_AGE]
16    if COL_HT in work.columns: work["Ht"] = work[COL_HT]
17    if COL_WT in work.columns: work["Wt"] = work[COL_WT]
18
19    work, beta_hw = hw_orthogonalize(work)
20
21    work["Y_star"] = sv(work[COL_Y].values)
22    work["logitY"] = np.log(work["Y_star"]/(1.0 - work["Y_star"]))
23    work["GA_c"] = work["GA"] - work["GA"].mean()
24    work["BMI_c"] = work["BMI"] - work["BMI"].mean()
25    work["rHT_c"] = work["rHT"] - work["rHT"].mean()
26    work["rWT_c"] = work["rWT"] - work["rWT"].mean()
27    if "Age" in work.columns: work["Age_c"] = work["Age"] - work["Age"].mean()
28

```



```

29 formula = f"1 + cr(GA_c, df={DF_GA}) * cr(BMI_c, df={DF_BMI})" \
30         f" + cr(rHT_c, df={DF_RHT}) + cr(rWT_c, df={DF_RWT})"
31 if "Age_c" in work.columns:
32     formula += f" + cr(Age_c, df={DF_AGE})"
33
34 try:
35     X = dmatrix(formula, work, return_type="dataframe")
36     md = sm.MixedLM(work["logitY"].values, X.values,
37                     groups=work[COL_ID].astype(str).values,
38                     exog_re=np.ones((len(work),1)))
39     res = md.fit(reml=False, method="lbfgs")
40 except Exception as e:
41     raise RuntimeError(f"GAMM 拟合失败: {e}\n公式: {formula}")
42
43 resid_var = float(res.scale)
44 try:
45     group_var = float(res.cov_re.iloc[0,0]) if hasattr(res,"cov_re") else
float(res.cov_re[0,0])
46 except Exception:
47     group_var = float(np.squeeze(np.array(res.cov_re)))
48
49 print(f"[GAMM] 公式: {formula}")
50 print(f"[GAMM] 残差方差={resid_var:.4f}, 组方差={group_var:.4f}")
51 return res, X.design_info, work, resid_var, group_var, beta_hw
52
53
54 #构造个体F_i(t)
55 def build_F_from_gamm(sub, res, design_info, work, t_grid, beta_hw):
56     GA_mean = work["GA"].mean()
57     BMI_mean = work["BMI"].mean()
58     Age_mean = work["Age"].mean() if "Age" in work.columns else None
59     rHT_mean = work["rHT"].mean()
60     rWT_mean = work["rWT"].mean()
61     thr = logit(Y_THRESH)
62
63     N = len(sub); T = len(t_grid)
64     F_mat = np.zeros((N, T), dtype=float)
65
66     for idx, row in enumerate(sub.itertuples(index=False)):
67         bmi0 = getattr(row, "BMI0")
68         age0 = getattr(row, COL_AGE) if (COL_AGE in sub.columns) else None
69         ht0 = getattr(row, COL_HT) if (COL_HT in sub.columns) else None

```

```

70     wt0 = getattr(row, COL_WT) if (COL_WT in sub.columns) else None
71
72     rht_vec, rwt_vec = hw_orth_apply(bmi0, ht0, wt0, beta_hw, len(t_grid))
73
74     new = {
75         "GA_c": (t_grid - GA_mean),
76         "BMI_c": np.full_like(t_grid, bmi0 - BMI_mean),
77         "rHT_c": (rht_vec - rHT_mean),
78         "rWT_c": (rwt_vec - rWT_mean)
79     }
80     if "Age" in work.columns:
81         new["Age_c"] = np.full_like(t_grid, (age0 if age0 is not None else
82         Age_mean) - Age_mean)
83
84     mu = predict_mu(design_info, res, new)
85     p = inv_logit(mu - thr)
86     F = np.maximum.accumulate(np.clip(p, 0.0, 1.0))
87     F_mat[idx,:] = F
88     return F_mat
89
90 #DP (风险段成本)
91 def build_time_grid():
92     t0 = 0.0
93     t_grid = np.arange(t0, T_MAX + 1e-9, DT)
94     t_candidates = np.arange(WIN_MIN, WIN_MAX + 1e-9, DT)
95     idx = lambda t: int(round((t - t0)/DT))
96     tk_index = {}
97     for t in t_candidates:
98         items = []
99         tk = t; prev = 0.0
100         while tk <= T_MAX + 1e-9:
101             items.append((idx(tk), idx(prev), stage_weight(tk)))
102             prev = tk; tk += DELTA
103         tk_index[t] = items
104     return t_grid, t_candidates, tk_index
105
106
107 def dp_partition_risk(sub_sorted, F_sorted, t_grid, t_candidates, tk_index,
108                       gmax=G_MAX, nmin=N_MIN_GROUP, gamma=GAMMA_STRUCT):
109     N, T = F_sorted.shape
110     prefix = np.vstack([np.zeros((1,T)), np.cumsum(F_sorted, axis=0)])

```

```

111 def seg_sum(i,j,ti): return prefix[j+1,ti] - prefix[i,ti]
112
113 INF = 1e100
114 cost = np.full((N,N), np.inf, float)
115 tstar_idx = np.full((N,N), -1, int)
116
117 for i in range(N):
118     for j in range(i,N):
119         if (j-i+1) < nmin: continue
120         best_val = INF; best_idx = -1
121         n_g = (j-i+1)
122         for t in t_candidates:
123             acc = 0.0
124             for (it, ip, wt) in tk_index[t]:
125                 acc += wt * (seg_sum(i,j,it) - seg_sum(i,j,ip))
126                 it0 = int(round((t-0.0)/DT))
127                 acc += (1.0 - seg_sum(i,j,it0)/max(n_g,1)) * n_g * LAMBDA_FAIL
128                 if acc < best_val:
129                     best_val = acc; best_idx = it0
130             cost[i,j] = best_val; tstar_idx[i,j] = best_idx
131
132 DP = np.full((gmax+1, N), INF, float); PREV = np.full((gmax+1, N), -1, int
)
133 for j in range(N): DP[1,j] = cost[0,j]; PREV[1,j] = -1
134 for k in range(2,gmax+1):
135     for j in range(k-1,N):
136         best=INF; best_i=-1
137         for i in range(k-1,j+1):
138             v = DP[k-1,i-1] + cost[i,j]
139             if v < best: best=v; best_i=i
140         DP[k,j]=best; PREV[k,j]=best_i
141
142 logN = np.log(N)
143 crit_list=[]
144 for k in range(1,gmax+1):
145     raw = DP[k,N-1]; pen = gamma*(k-1)*logN
146     crit_list.append((k, raw, raw+pen))
147     print(f"  k={k}: 原始成本={raw:.2f}, 惩罚={pen:.2f}, 总准则={raw+pen
:.2f}")
148 best_k, _, _ = min(crit_list, key=lambda x: x[2])
149
150 segs=[]; j=N-1; k=best_k

```

```

151     while k>=1:
152         i=PREV[k,j]; i=0 if i==-1 else i
153         segs.append((i,j)); j=i-1; k-=1
154     segs=list(reversed(segs))
155     return best_k, segs, crit_list, cost, tstar_idx
156
157
158 def summarize_groups(sub_sorted, F_sorted, segs, t_grid, tstar_idx, tk_index):
159     rows=[]
160     for g,(s,e) in enumerate(segs, start=1):
161         it=tstar_idx[s,e]; t_star=t_grid[it]
162         Fg = F_sorted[s:e+1,:]
163         one_shot=float(Fg[:,it].mean()); retest=1.0-one_shot
164         A=Fg.sum(axis=0); acc=0.0
165         for (itk, ip, wt) in tk_index[t_star]:
166             acc += wt*(A[itk]-A[ip])
167         acc += (1.0 - A[it]/max((e-s+1),1))*(e-s+1)*LAMBDA_FAIL
168         rows.append({
169             "组别":g, "人数":(e-s+1),
170             "BMI下界":float(sub_sorted["BMI0"].iloc[s]),
171             "BMI上界":float(sub_sorted["BMI0"].iloc[e]),
172             "最佳检测时点(周)":float(t_star),
173             "一次达标率":one_shot, "重测率":retest, "风险值":float(acc)
174         })
175     return pd.DataFrame(rows)

```

0.7 这是问题四所用核心代码，特征提取并输入到所用的两个机器学习模型中进行训练。通过重采样方法解决类别不平衡的问题。

```

1
2 ### 问题4_核心代码
3
4
5
6 # 机器学习相关
7 from sklearn.model_selection import (
8     train_test_split, cross_val_score, GridSearchCV,
9     StratifiedKFold, cross_validate, GroupKFold
10 )
11 from sklearn.ensemble import (
12     GradientBoostingClassifier, AdaBoostClassifier,
13     RandomForestClassifier, ExtraTreesClassifier
14 )

```

```

15 from sklearn.metrics import (
16     roc_auc_score, average_precision_score, roc_curve,
17     precision_recall_curve, accuracy_score, precision_score,
18     recall_score, f1_score, confusion_matrix, classification_report,
19     brier_score_loss, log_loss, matthews_corrcoef, cohen_kappa_score
20 )
21 from sklearn.preprocessing import StandardScaler
22 from sklearn.impute import SimpleImputer
23
24 # 重采样
25 from imblearn.over_sampling import (
26     SMOTE, BorderlineSMOTE, SVMSMOTE, RandomOverSampler, ADASYN
27 )
28 from imblearn.under_sampling import RandomUnderSampler
29 from imblearn.combine import SMOTEENN, SMOTETomek
30
31 import joblib
32 warnings.filterwarnings('ignore')
33
34 # 设置随机种子
35 RANDOM_STATE = 42
36 np.random.seed(RANDOM_STATE)
37
38 _MODEL_ZH = {
39     "GradientBoosting": "梯度提升(GBDT)",
40     "AdaBoost": "AdaBoost"
41 }
42
43 _DATASET_ZH = {
44     "original": "原始数据",
45     "random_oversample": "随机过采样",
46     "random_undersample": "随机欠采样",
47     "smote": "SMOTE",
48     "borderline_smote": "边界SMOTE",
49     "smoteenn": "SMOTEENN",
50 }
51
52 _METRIC_ZH = {
53     "AUC": "AUC",
54     "F1": "F1",
55     "Recall": "召回率",
56     "Precision": "精确率",
57     "MCC": "MCC",
58     "Specificity": "特异性",

```

```

57 }
58 def _zh_model(name): return _MODEL_ZH.get(name, name)
59 def _zh_dataset(name): return _DATASET_ZH.get(name, name)
60 def _zh_metric(name): return _METRIC_ZH.get(name, name)
61
62
63 class AneuploidyDetector:
64     def __init__(self, config=None):
65         self.config = config or self._get_default_config()
66         self.models = {}
67         self.results = {}
68         self.feature_names = None
69         self.sample_weights = None
70         self.pregnant_groups = None
71         self._setup_directories()
72
73     def _get_default_config(self):
74         return {
75             'input_file': './附件-女胎处理_01处理.xlsx',
76             'sheet_name': '附件-女胎处理',
77             'abnormal_col': '染色体的非整倍体',
78             'pregnant_id_col': '孕妇代码', # 孕妇ID列
79             'weight_col': 'w_Q', # 权重列
80             'output_dir': Path('./gbdt_enhanced_results'),
81             'test_ratio': 0.3,
82             'cv_folds': 5,
83             'random_state': RANDOM_STATE
84         }
85
86     def _setup_directories(self):
87         self.config['output_dir'].mkdir(parents=True, exist_ok=True)
88         (self.config['output_dir'] / 'figures').mkdir(exist_ok=True)
89         (self.config['output_dir'] / 'models').mkdir(exist_ok=True)
90         (self.config['output_dir'] / 'reports').mkdir(exist_ok=True)
91
92     def load_and_process_data(self):
93         df = pd.read_excel(self.config['input_file'],
94                             sheet_name=self.config['sheet_name'])
95         print(f"数据加载完成: {len(df)} 个样本")
96         abnormal_col = self.config['abnormal_col']
97         if abnormal_col in df.columns:
98             # 直接将染色体的非整倍体列作为标签 (0=正常, 1=异常)

```

```

99         df['label'] = pd.to_numeric(df[abnormal_col], errors='coerce').
fillna(0).astype(int)
100         # 统计
101         n_normal = (df['label'] == 0).sum()
102         n_abnormal = (df['label'] == 1).sum()
103         n_pregnant = df[self.config['pregnant_id_col']].nunique()
104
105         df.to_excel(self.config['output_dir'] / 'processed_data.xlsx',
106                     index=False)
107     else:
108         raise ValueError(f"未找到标签列: {abnormal_col}")
109
110     self.data = df
111     return df
112
113     def prepare_features(self, df):
114         exclude_cols = ['label', '异常标签', self.config['pregnant_id_col'],
115                         self.config['abnormal_col'], self.config['weight_col']
116         ']]
117         numeric_cols = []
118         for col in df.columns:
119             if col not in exclude_cols:
120                 try:
121                     numeric_data = pd.to_numeric(df[col], errors='coerce')
122                     if numeric_data.notna().mean() > 0.3:
123                         numeric_cols.append(col)
124                 except Exception:
125                     pass
126         print(f"\n自动识别 {len(numeric_cols)} 个数值特征")
127
128         X = df[numeric_cols].copy()
129         for col in X.columns:
130             X[col] = pd.to_numeric(X[col], errors='coerce')
131         X = X.dropna(axis=1, how='all')
132         valid_cols = X.columns.tolist()
133         print(f"有效特征: {len(valid_cols)} 个")
134
135         imputer = SimpleImputer(strategy='median')
136         X_imputed = imputer.fit_transform(X)
137         X = pd.DataFrame(X_imputed, columns=valid_cols, index=df.index)
138
139         self.feature_names = valid_cols

```

```

139     y = df['label'].values
140
141     if self.config['weight_col'] in df.columns:
142         weights = pd.to_numeric(df[self.config['weight_col']], errors='
coerce').fillna(1.0).values
143         print(f" 使用样本权重列: {self.config['weight_col']}")
144         print(f" 权重范围: [{weights.min():.3f}, {weights.max():.3f}]")
145         print(f" 平均权重: {weights.mean():.3f}")
146     else:
147         print(" 未找到权重列, 使用默认权重1.0")
148         weights = np.ones(len(y))
149
150     groups = df[self.config['pregnant_id_col']].values
151     print(f" 使用孕妇分组列: {self.config['pregnant_id_col']}")
152
153     unique, counts = np.unique(y, return_counts=True)
154     print(f" 标签分布: {dict(zip(unique, counts))}")
155     return X, y, weights, groups
156
157 def split_by_pregnant(self, X, y, weights, groups, test_size=0.3):
158     print("\n按孕妇分组切分数据...")
159     unique_groups = np.unique(groups)
160     group_labels = []
161     for group_id in unique_groups:
162         group_mask = groups == group_id
163         group_y = y[group_mask]
164         group_label = int(np.median(group_y))
165         group_labels.append(group_label)
166     group_labels = np.array(group_labels)
167
168     train_groups, test_groups = train_test_split(
169         unique_groups, test_size=test_size,
170         random_state=self.config['random_state'],
171         stratify=group_labels
172     )
173
174     train_mask = np.isin(groups, train_groups)
175     test_mask = np.isin(groups, test_groups)
176
177     X_train = X[train_mask]; X_test = X[test_mask]
178     y_train = y[train_mask]; y_test = y[test_mask]
179     weights_train = weights[train_mask]; weights_test = weights[test_mask]

```



```

180     groups_train = groups[train_mask]; groups_test = groups[test_mask]
181
182     print(f"  训练集: {len(X_train)} 样本, {len(train_groups)} 位孕妇")
183     print(f"  测试集: {len(X_test)} 样本, {len(test_groups)} 位孕妇")
184     print(f"  训练集异常率: {(y_train==1).mean()*100:.2f}%")
185     print(f"  测试集异常率: {(y_test==1).mean()*100:.2f}%")
186
187     overlap = set(groups_train) & set(groups_test)
188     if len(overlap) > 0:
189         print(f"  警告: 有 {len(overlap)} 位孕妇同时出现在训练集和测试集!")
190     ")
191
192     else:
193         print("  确认: 没有孕妇同时出现在训练集和测试集")
194
195     return (X_train, y_train, weights_train, groups_train,
196            X_test, y_test, weights_test, groups_test)
197
198 def create_resampled_datasets(self, X, y, weights, groups):
199     print("\n" + "="*60)
200     print("步骤2: 生成重采样数据集")
201     print("="*60)
202
203     datasets = {'original': (X, y, weights, groups)}
204
205     unique, counts = np.unique(y, return_counts=True)
206     if len(unique) < 2:
207         print("  数据只有一个类别, 无法进行重采样")
208         return datasets
209
210     min_class_count = min(counts)
211     print(f"  少数类样本数: {min_class_count}")
212
213     resamplers = {}
214     if min_class_count >= 2:
215         resamplers['random_oversample'] = RandomOverSampler(random_state=
RANDOM_STATE)
216
217         resamplers['random_undersample'] = RandomUnderSampler(random_state
=RANDOM_STATE)
218
219         if min_class_count >= 6:
220             k_neighbors = min(5, min_class_count - 1)
221             resamplers['smote'] = SMOTE(random_state=RANDOM_STATE,
k_neighbors=k_neighbors)

```

```

219         resamplers['borderline_smote'] = BorderlineSMOTE(random_state=
RANDOM_STATE,
220                                                         k_neighbors=
k_neighbors)
221         resamplers['smoteenn'] = SMOTEENN(random_state=RANDOM_STATE,
222                                             smote=SMOTE(k_neighbors=
k_neighbors))
223
224     for name, resampler in resamplers.items():
225         try:
226             if 'smote' in name.lower() or 'enn' in name.lower():
227                 X_with_weight = X.copy()
228                 X_with_weight['weight'] = weights
229                 X_res, y_res = resampler.fit_resample(X_with_weight, y)
230                 weights_res = X_res['weight'].values
231                 X_res = X_res.drop(['weight'], axis=1)
232                 n_resampled = len(y_res)
233                 if 'enn' in name.lower():
234                     groups_res = np.array([f'synthetic_{i}' for i in range
(n_resampled)])
235             else:
236                 n_original = len(y)
237                 groups_res = np.array(['synthetic'] * n_resampled)
238                 if n_resampled >= n_original:
239                     groups_res[:n_original] = groups
240             else:
241                 indices = np.arange(len(y))
242                 indices_res, y_res = resampler.fit_resample(indices.
reshape(-1, 1), y)
243                 indices_res = indices_res.ravel()
244                 X_res = X.iloc[indices_res].reset_index(drop=True)
245                 weights_res = weights[indices_res]
246                 groups_res = groups[indices_res]
247
248                 datasets[name] = (X_res, y_res, weights_res, groups_res)
249                 n_normal = int(np.sum(y_res == 0))
250                 n_abnormal = int(np.sum(y_res == 1))
251                 ratio = n_abnormal / n_normal if n_normal > 0 else 0
252                 print(f" {name:20s}: 正常={n_normal:4d}, 异常={n_abnormal:4d
}, 比例={ratio:.3f}")
253         except Exception as e:
254             print(f" {name:20s}: 失败 - {str(e)}")

```

```

255         stats_df = self._create_resample_stats(datasets)
256         stats_df.to_excel(self.config['output_dir'] / 'resample_stats.xlsx',
257         index=False)
258         return datasets
259
260     def _create_resample_stats(self, datasets):
261         stats = []
262         for name, data_tuple in datasets.items():
263             y = data_tuple[1]
264             n_normal = int(np.sum(y == 0))
265             n_abnormal = int(np.sum(y == 1))
266             total = len(y)
267             pos_ratio = n_abnormal / total * 100 if total > 0 else 0
268             class_ratio = n_abnormal / n_normal if n_normal > 0 else 0
269             stats.append({
270                 'Dataset': name,
271                 'Normal': n_normal,
272                 'Abnormal': n_abnormal,
273                 'Total': total,
274                 'Positive_Rate': pos_ratio,
275                 'Class_Ratio': class_ratio
276             })
277         return pd.DataFrame(stats)
278
279     def train_models(self, datasets):
280         print("\n" + "="*60)
281         print("步骤3: 模型训练与评估")
282         print("="*60)
283
284         model_configs = {
285             'GradientBoosting': {
286                 'model': GradientBoostingClassifier(random_state=RANDOM_STATE)
287             },
288             'param_grid': {
289                 'n_estimators': [50, 100, 200],
290                 'learning_rate': [0.01, 0.05, 0.1],
291                 'max_depth': [3, 5, 7],
292                 'min_samples_split': [10, 20],
293                 'min_samples_leaf': [5, 10],
294                 'subsample': [0.8, 1.0]
295             }
296         }

```

```

295         },
296         'AdaBoost': {
297             'model': AdaBoostClassifier(random_state=RANDOM_STATE,
algorithm='SAMME'),
298             'param_grid': {
299                 'n_estimators': [50, 100, 200],
300                 'learning_rate': [0.01, 0.05, 0.1, 0.5]
301             }
302         }
303     }
304     results = {}
305
306     for data_name, data_tuple in datasets.items():
307         print(f"\n处理数据集: {data_name}")
308         print("-" * 40)
309
310         X = data_tuple[0]
311         y = data_tuple[1]
312         weights = data_tuple[2] if len(data_tuple) > 2 else np.ones(len(y)
)
313         groups = data_tuple[3] if len(data_tuple) > 3 else np.arange(len(y)
))
314
315         (X_train, y_train, weights_train, groups_train,
316          X_test, y_test, weights_test, groups_test) = self.
split_by_pregnant(
317             X, y, weights, groups, test_size=self.config['test_ratio']
318         )
319
320         scaler = StandardScaler()
321         X_train_scaled = scaler.fit_transform(X_train)
322         X_test_scaled = scaler.transform(X_test)
323
324         data_results = {}
325
326         for model_name, config in model_configs.items():
327             print(f"  训练 {model_name}...")
328
329             n_unique_groups = len(np.unique(groups_train))
330             n_splits = min(self.config['cv_folds'], n_unique_groups)
331
332             cv = GroupKFold(n_splits=n_splits)

```

```

333         groups_for_cv = groups_train
334
335     if len(y_train) < 100:
336         simple_grid = {}
337         for key, values in config['param_grid'].items():
338             if key == 'n_estimators':
339                 simple_grid[key] = [50, 100]
340             elif key == 'learning_rate':
341                 simple_grid[key] = [0.05, 0.1]
342             elif key == 'max_depth':
343                 simple_grid[key] = [3, 5]
344             else:
345                 simple_grid[key] = values[:2] if len(values) > 1
346
347     else values
348         param_grid = simple_grid
349     else:
350         param_grid = config['param_grid']
351
352     def weighted_auc_scorer(estimator, X_, y_):
353         y_pred_ = estimator.predict_proba(X_)[:, 1]
354         return roc_auc_score(y_, y_pred_)
355
356     grid = GridSearchCV(
357         config['model'], param_grid,
358         cv=cv if groups_for_cv is None else cv.split(
359             X_train_scaled, y_train,
360                                     groups_for_cv
361         ),
362         scoring=weighted_auc_scorer,
363         n_jobs=-1, verbose=0
364     )
365
366     try:
367         grid.fit(X_train_scaled, y_train, sample_weight=
368             weights_train)
369         best_model = grid.best_estimator_
370
371         train_metrics = self._evaluate_model(best_model,
372             X_train_scaled,
373                                     y_train,
374             weights_train)
375         test_metrics = self._evaluate_model(best_model,

```

```

X_test_scaled,
369                                     y_test, weights_test)
370
371     y_test_proba = best_model.predict_proba(X_test_scaled)[: ,
1]
372
373     data_results[model_name] = {
374         'model': best_model,
375         'scaler': scaler,
376         'train_metrics': train_metrics,
377         'test_metrics': test_metrics,
378         'best_params': grid.best_params_,
379         'cv_score': grid.best_score_,
380         'X_test': X_test_scaled,
381         'y_test': y_test,
382         'y_test_proba': y_test_proba,
383         'weights_test': weights_test,
384         'groups_test': groups_test
385     }
386
387     print(f"      CV AUC: {grid.best_score_:.4f}")
388     print(f"      测试 AUC: {test_metrics['auc']:.4f}, "
389           f"F1: {test_metrics['f1']:.4f}, "
390           f"召回率: {test_metrics['recall']:.4f}")
391
392     except Exception as e:
393         print(f"      训练失败: {str(e)}")
394
395     results[data_name] = data_results
396
397     self.results = results
398     return results
399
400 def _evaluate_model(self, model, X, y, sample_weight=None):
401     y_pred = model.predict(X)
402     y_proba = model.predict_proba(X)[: , 1]
403
404     try:
405         mcc = matthews_corrcoef(y, y_pred, sample_weight=sample_weight)
406     except TypeError:
407         mcc = matthews_corrcoef(y, y_pred)
408     try:

```

```

409         kappa = cohen_kappa_score(y, y_pred, sample_weight=sample_weight)
410     except TypeError:
411         kappa = cohen_kappa_score(y, y_pred)
412
413     metrics = {
414         'accuracy': accuracy_score(y, y_pred, sample_weight=sample_weight)
415     ,
416         'precision': precision_score(y, y_pred, sample_weight=
sample_weight, zero_division=0),
417         'recall': recall_score(y, y_pred, sample_weight=sample_weight),
418         'f1': f1_score(y, y_pred, sample_weight=sample_weight),
419         'auc': roc_auc_score(y, y_proba, sample_weight=sample_weight),
420         'ap': average_precision_score(y, y_proba, sample_weight=
sample_weight),
421         'mcc': mcc,
422         'kappa': kappa,
423         'brier': brier_score_loss(y, y_proba, sample_weight=sample_weight)
424     }
425
426     cm = confusion_matrix(y, y_pred, sample_weight=sample_weight)
427     if cm.shape == (2, 2):
428         tn, fp, fn, tp = cm.ravel()
429         metrics['specificity'] = tn / (tn + fp) if (tn + fp) > 0 else 0
430         metrics['npv'] = tn / (tn + fn) if (tn + fn) > 0 else 0
431
432     return metrics
433
434     def plot_comparison(self):
435         print("\n" + "="*60)
436         print("步骤4: 生成可视化报告")
437         print("="*60)
438
439         self._plot_metrics_comparison()
440         self._plot_roc_curves()
441         self._plot_best_model_analysis()
442         self._plot_weight_analysis()
443
444         print("  可视化报告已生成")
445
446     def _plot_weight_analysis(self):
447         if not hasattr(self, 'results') or not self.results:

```

```

448
449     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
450
451     best_result = None
452     for data_name, models in self.results.items():
453         for model_name, result in models.items():
454             if 'weights_test' in result:
455                 best_result = result
456                 break
457         if best_result:
458             break
459
460     if best_result and 'weights_test' in best_result:
461         weights = best_result['weights_test']
462         y_test = best_result['y_test']
463         y_proba = best_result['y_test_proba']
464
465         axes[0].hist(weights[y_test == 0], bins=20, alpha=0.5, label='正常
', density=True)
466         axes[0].hist(weights[y_test == 1], bins=20, alpha=0.5, label='异常
', density=True)
467         axes[0].set_xlabel('样本权重 (w_Q)')
468         axes[0].set_ylabel('密度')
469         axes[0].set_title('按类别的权重分布')
470         axes[0].legend()
471         axes[0].grid(True, alpha=0.3)
472
473         scatter = axes[1].scatter(weights, y_proba, c=y_test, cmap='
coolwarm', alpha=0.6)
474         axes[1].set_xlabel('样本权重 (w_Q)')
475         axes[1].set_ylabel('预测概率')
476         axes[1].set_title('权重与预测的关系')
477         plt.colorbar(scatter, ax=axes[1], label='真实标签')
478         axes[1].grid(True, alpha=0.3)
479
480     plt.suptitle('样本权重分析', fontsize=14, y=1.02)
481     plt.tight_layout()
482     plt.savefig(self.config['output_dir'] / 'figures' / 'weight_analysis.
png',
483                 dpi=200, bbox_inches='tight')
484     plt.close()
485

```



```

486 def _plot_metrics_comparison(self):
487     import numpy as np
488     import pandas as pd
489     import matplotlib as mpl
490     import matplotlib.pyplot as plt
491     import matplotlib.ticker as mticker
492     from matplotlib.patches import Patch
493
494     mpl.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial
Unicode MS',
495                                         'DejaVu Sans']
496     mpl.rcParams['axes.unicode_minus'] = False
497     mpl.rcParams.update({
498         'axes.facecolor': 'white',
499         'figure.facecolor': 'white',
500         'axes.edgecolor': '#E0E0E0',
501         'axes.linewidth': 1.0,
502         'axes.titlesize': 12,
503         'axes.titleweight': 'bold',
504         'axes.labelsiz e': 11,
505         'xtick.labelsiz e': 10,
506         'ytick.labelsiz e': 10,
507         'legend.fontsiz e': 10,
508         'grid.color': '#E6E6E6',
509         'grid.linestyle': ':',
510         'grid.linewidth': 0.8,
511     })
512
513     comparison_data = []
514     for data_name, models in self.results.items():
515         for model_name, result in models.items():
516             comparison_data.append({
517                 'Dataset': data_name,
518                 'Model': model_name,
519                 'AUC': result['test_metrics']['auc'],
520                 'F1': result['test_metrics']['f1'],
521                 'Recall': result['test_metrics']['recall'],
522                 'Precision': result['test_metrics']['precision'],
523                 'MCC': result['test_metrics']['mcc']
524             })
525     df_comp = pd.DataFrame(comparison_data)
526

```

```

527     # 固定数据集/模型顺序
528     dataset_order = list(pd.unique(df_comp['Dataset']))
529     model_order_raw = list(pd.unique(df_comp['Model']))
530     df_comp['Dataset'] = pd.Categorical(df_comp['Dataset'], categories=
dataset_order, ordered=True)
531
532     display_model_order = [_zh_model(m) for m in model_order_raw]
533     cmap = plt.get_cmap('tab10')
534     color_map = {m_disp: cmap(i) for i, m_disp in enumerate(
display_model_order)}
535     hatches_seq = ['/', '\\\\', 'x', '.', '-', '+', 'o', '0', '*']
536     hatch_map = {m_disp: hatches_seq[i % len(hatches_seq)] for i, m_disp
in enumerate(display_model_order)}
537
538     fig, axes = plt.subplots(2, 3, figsize=(16, 10))
539     axes = axes.flatten()
540     metrics = ['AUC', 'F1', 'Recall', 'Precision', 'MCC']
541
542     def annotate_bars(ax, fmt='{:.2f}'):
543         for container in ax.containers:
544             ax.bar_label(container, fmt=fmt, padding=2, fontsize=9)
545
546
547     for idx, metric in enumerate(metrics):
548         ax = axes[idx]
549         pivot_df = df_comp.pivot(index='Dataset', columns='Model', values=
metric)
550
551         pivot_df = pivot_df.reindex(columns=model_order_raw)
552         pivot_df.index = [_zh_dataset(i) for i in pivot_df.index]
553         rename_map = {orig: disp for orig, disp in zip(model_order_raw,
display_model_order)}
554         pivot_df = pivot_df.rename(columns=rename_map)
555         col_colors = [color_map[c] for c in pivot_df.columns]
556         bars = pivot_df.plot(kind='bar', ax=ax, width=0.76,
557                               color=col_colors, edgecolor='#444', linewidth
=0.8, zorder=2)
558         for j, container in enumerate(ax.containers):
559             for p in container.patches:
560                 p.set_hatch(hatch_map[pivot_df.columns[j]])
561         ax.set_title(f'_{zh_metric(metric)}对比')
562         ax.set_xlabel('')

```

```

563         ax.grid(axis='y', alpha=0.6, zorder=0)
564         for spine in ['top', 'right']:
565             ax.spines[spine].set_visible(False)
566         ax.set_xticklabels(ax.get_xticklabels(), rotation=18, ha='right')
567
568         if metric == 'MCC':
569             ax.set_ylim(-1.0, 1.0)
570             ax.yaxis.set_major_locator(mticker.MultipleLocator(0.5))
571         else:
572             ax.set_ylim(0.0, 1.0)
573             ax.yaxis.set_major_locator(mticker.MultipleLocator(0.2))
574             ax.yaxis.set_major_formatter(mticker.FormatStrFormatter('%.1f'))
575             annotate_bars(ax, fmt='{:.2f}')
576             ax.legend_.remove()
577         axes[-1].axis('off')
578         legend_handles = [Patch(facecolor=color_map[m], edgecolor='#444',
579                                hatch=hatch_map[m], label=m) for m in
display_model_order]
580         fig.legend(handles=legend_handles, loc='upper center',
581                   ncol=min(4, len(legend_handles)), frameon=True, fancybox=True,
582                   framealpha=0.9, borderpad=0.4, bbox_to_anchor=(0.5, 1.01))
583         plt.suptitle('不同数据集的模型性能对比', fontsize=16, weight='bold', y
=1.06)
584         plt.tight_layout(rect=[0.02, 0.02, 0.98, 0.98])
585         out_dir = self.config['output_dir'] / 'figures'
586         out_dir.mkdir(parents=True, exist_ok=True)
587         plt.savefig(out_dir / 'metrics_comparison.png', dpi=300, bbox_inches='
tight')
588         plt.close()
589
590     def _plot_roc_curves(self):
591         fig, axes = plt.subplots(2, 3, figsize=(15, 10))
592         dataset_names = list(self.results.keys())
593         for idx, data_name in enumerate(dataset_names[:6]):
594             ax = axes[idx // 3, idx % 3]
595             if data_name in self.results:
596                 for model_name, result in self.results[data_name].items():
597                     y_test = result['y_test']
598                     y_proba = result['y_test_proba']
599                     weights_test = result.get('weights_test', None)
600                     fpr, tpr, _ = roc_curve(y_test, y_proba, sample_weight=
weights_test)

```

```

601         auc = roc_auc_score(y_test, y_proba, sample_weight=
weights_test)
602         ax.plot(fpr, tpr, label=f'_{zh_model(model_name)} (AUC={
auc:.3f})', lw=2)
603         ax.plot([0, 1], [0, 1], 'k--', lw=1, alpha=0.5)
604         ax.set_xlabel('假阳性率')
605         ax.set_ylabel('真阳性率')
606         ax.set_title(f'ROC曲线 - {_zh_dataset(data_name)}')
607         ax.legend(loc='lower right')
608         ax.grid(True, alpha=0.3)
609     for idx in range(len(dataset_names), 6):
610         fig.delaxes(axes[idx // 3, idx % 3])
611
612     plt.suptitle('不同数据集（加权）的ROC曲线', fontsize=14, y=1.02)
613     plt.tight_layout()
614     plt.savefig(self.config['output_dir'] / 'figures' / 'roc_curves.png',
615                 dpi=200, bbox_inches='tight')
616     plt.close()

```